EVENTFORCE MANAGEMENT SYSTEM-SALESFORCE IMPLEMENTATION

SREE SASTHA INSTITUTE OF ENGINEERING AND TECHNOLOGY

TEAM ID : SWTID-2026-8676

TEAM SIZE : 5

TEAM LEADER : SHOBANA S

EMAIL ID : shobanashri77@gmail.com

TEAM MEMBER : GOKUL SAKTHI V

EMAIL ID : gokulsakthi633@gmail.com

TEAM MEMBER : HARISH V

EMAIL ID : harishvijayarangan.v@gmail.com

TEAM MEMBER : SATHIYA M

EMAIL ID : sathiyamohan2678@gmail.com

TEAM MEMBER : SNEKHA M

EMAIL ID : snekhamoorthi2006@gmail.com

USER STORY :

Currently, event planning operations—including client bookings, venue reservations, and vendor coordination—are fragmented, leading to communication gaps and scheduling conflicts.

The **EventForce Management System** resolves this by providing a unified Salesforce solution. By leveraging a structured relational data model, the system enables event teams to seamlessly manage **Events, Clients, Vendors, Venues,** and **Feedback** from a single workspace. It eliminates manual oversight through automated Record-Triggered Flows that handle client reminders, and enforces strict governance via Approval Processes for event cancellations. Furthermore, programmatic Apex logic safeguards operations by actively preventing venue double-bookings and dynamically updating availability status. Finally, the system empowers leadership with real-time operational visibility through a dedicated Lightning App, comprehensive Reports, and dynamic Dashboards.

1. INTRODUCTION

1.1 Project Overview

The EventForce Management System represents a comprehensive digital transformation initiative specifically designed to modernize and scale the operations of Event Planner Solutions. Historically, event management requires the orchestration of countless moving parts—from tracking client communications and securing venue spaces to coordinating multiple third-party vendors and processing post-event feedback. When managed through fragmented systems, manual spreadsheets, and disconnected communication channels, these processes become highly susceptible to human error, double bookings, and operational bottlenecks. The EventForce Management System resolves these challenges by centralizing the entire event lifecycle within a highly structured, cloud-based Salesforce Customer Relationship Management (CRM) platform.

At its core, the system acts as a single source of truth for the organization. It seamlessly manages a complex ecosystem of core entities: Clients, Events, Venues, Vendors, and Feedback. By utilizing a robust relational data model—including a custom junction object that allows multiple vendors to be assigned to a single

event—the CRM provides users with a 360-degree view of every upcoming, ongoing, and completed event.

To achieve a high degree of automation and enforce strict business logic, the solution leverages a hybrid architectural approach that combines Salesforce's powerful declarative tools with advanced programmatic features:

Declarative Automation ("Clicks Not Code"): The system heavily utilizes standard Salesforce administrative tools to drive efficiency. Formula fields automatically calculate event budgets based on the designated event type (e.g., Corporate vs. Wedding). Validation rules enforce strict data formatting, such as ensuring correct email structures. Furthermore, Record-Triggered Flows are deployed to automate routine communications, such as dispatching customized email reminders to clients exactly three days before their scheduled event. The system also features a meticulously designed Approval Process that systematically routes event cancellation requests to management, ensuring that changes to the event schedule are reviewed, controlled, and communicated to all stakeholders automatically.

Programmatic Engineering (Custom Code): For complex business requirements that exceed the capabilities of standard declarative tools, the system employs custom Apex programming. Apex Classes and Triggers operate behind the scenes to execute advanced physical inventory logic. Most notably, custom programmatic logic is responsible for real-time venue availability management. By instantly querying the database upon record creation or modification, the system actively prevents the double booking of venues on the same date, guaranteeing that operational conflicts are intercepted before they can occur. Additionally, scheduled asynchronous Apex batches run nightly to automatically transition past events to a "Completed" status, keeping the database clean without requiring manual administrative effort.

Ultimately, this Salesforce implementation does more than just store data; it actively drives business value. By eradicating manual workflows, enforcing strict data integrity, and automating critical communications, the EventForce Management System is projected to reduce manual operational processes by 60% and improve overall booking efficiency by 40%, positioning Event Planner Solutions for sustainable, scalable growth.

1.2 Business Objectives and Value

The decision to implement the EventForce Management System is driven by a strategic imperative to transition Event Planner Solutions from a reactive, manually driven operational model to a proactive, highly automated enterprise environment. The primary objective of this Salesforce implementation is to construct a resilient technological foundation capable of automating key processes—such as event scheduling, cancellation approvals, and client reminders—while simultaneously providing executive and managerial levels with real-time visibility through dynamic reports and dashboards.

By successfully deploying this CRM solution, the organization is positioned to fundamentally transform its service delivery model. The business value derived from this project is categorized into four strategic pillars: Efficiency Gains, Customer Relationship Management (CRM) Excellence, Operational Collaboration, and Data-Driven Insights.

1.2.1 Core Operational Objectives

To understand the full value of the EventForce Management System, it is essential to examine the specific operational bottlenecks the system is designed to eliminate. The primary objectives focus on automating the most resource-intensive aspects of the event lifecycle:

- **Automated Event Scheduling and Validation:** Historically, scheduling required cross-referencing multiple calendars and spreadsheets, leading to human error and devastating double bookings. The objective here is to utilize programmatic logic (Apex) to instantly validate venue availability upon record creation, entirely removing the risk of scheduling conflicts and accelerating the booking timeline.

- **Systematized Cancellation Approvals:** Event cancellations are highly sensitive transactions that impact revenue, venue availability, and vendor schedules. The objective is to replace ad-hoc email requests with a structured Salesforce Approval Process. This ensures that every cancellation request is formally routed to management for review, and upon approval or rejection,

the system automatically cascades status updates and notifications to all impacted stakeholders.

- **Automated Client Reminders:** Missed communications lead to client anxiety and logistical failures. By utilizing Salesforce Record-Triggered Flows, the objective is to guarantee that customized, automated email reminders are dispatched to clients exactly prior to their events, ensuring high attendance, preparedness, and a polished brand image.

- **Real-Time Operational Visibility:** Operating without immediate access to data limits decision-making. The objective is to deploy customized dashboards that aggregate live data, allowing Event Administrators and Operations Managers to view current capacities, upcoming event volumes, and pending approvals at a single glance.

1.2.2 Pillar 1: Quantifiable Efficiency Gains

The most immediate and measurable impact of the EventForce Management System is the drastic reduction in administrative overhead. By replacing disparate tools with a single centralized platform, the business aims to achieve a **reduction of manual processes by 60%**.

This 60% reduction is realized by eliminating redundant data entry across different platforms. Event Coordinators no longer need to manually copy client details into venue reservation sheets or vendor assignment logs; entering the data once into the Client__c and Event__c objects propagates it throughout the system. Furthermore, automated budget calculations (via Formula Fields) and automated status updates eliminate hours of manual administrative calculation and record-keeping.

Beyond day-to-day data entry and automation, the EventForce Management System is engineered to deliver strategic business intelligence. The implementation includes customized Lightning App interfaces tailored to specific user roles, ensuring that Event Coordinators, Vendor Managers, and Administrators have secure, streamlined access to the tools they need. Through the deployment of dynamic Reports and real-time Dashboards, management gains immediate operational visibility. Leadership

can visually track the volume of upcoming events by month, monitor projected revenue pipelines, and assess vendor performance or client satisfaction at a glance.

Consequently, this reduction in manual labor directly correlates to an **improvement in booking efficiency by 40%**. With automation handling the administrative burden, Event Coordinators can process event inquiries, validate venue availability, assign vendors, and issue confirmations significantly faster. This rapid turnaround time not only allows the business to handle a higher volume of events with the same headcount but also secures revenue faster by reducing the time between initial client inquiry and final contract confirmation.

1.2.3 Pillar 2: Customer Relationship Management (CRM) Excellence

At the heart of the hospitality and event industry is the customer experience. The EventForce system is designed to deliver enhanced client satisfaction through a modernized CRM approach.

 Centralized Client Data: By maintaining comprehensive client profiles, the organization ensures that customers never have to repeat their preferences, contact details, or history. This 360-degree view allows event teams to provide highly personalized service, anticipating client needs based on historical data.

 Timely and Consistent Communications: Trust is built on reliability. By leveraging automated workflows to handle confirmations, status changes, and pre-event reminders, the system ensures that clients are perpetually kept in the loop. This consistent, branded communication removes the anxiety often associated with event planning, elevating the client's perception of the brand.

 Structured Feedback Collection: Improvement requires measurement. The implementation includes a dedicated Feedback__c object to systematically capture post-event ratings and comments. By digitizing and structuring this feedback, management can easily identify trends in customer satisfaction, address individual grievances promptly, and continuously refine their service offerings based on direct voice-of-the-customer data.

`

1.2.4 Pillar 3: Seamless Operational Collaboration

Event execution is inherently collaborative, requiring synchronized efforts across multiple distinct departments and external partners. The EventForce Management System aims to dismantle operational silos, fostering improved coordination among event coordinators, vendor managers, and clients.

1.2.5 Pillar 4: Data-Driven Insights and Operational Visibility

While seamless collaboration ensures smooth event execution, continuous improvement requires measurable insights. The EventForce Management System leverages native Salesforce Reports and Dashboards to transform raw operational data into actionable intelligence. By providing real-time visibility into upcoming event schedules, budget utilization, vendor performance, and client feedback, this pillar empowers management to make informed, strategic decisions that drive business growth and operational efficiency.

1.2.6 Pillar 5: Uncompromised Data Integrity and Security

As operational collaboration scales, protecting sensitive client and business data becomes paramount. This pillar focuses on enforcing strict data governance through comprehensive Validation Rules and advanced Apex logic—such as automated safeguards against venue double-bookings. Furthermore, it ensures that system access is tightly controlled via a robust architecture of Profiles, Roles, and Sharing Rules, guaranteeing that users only interact with the information essential to their specific duties.

2. IDEATION PHASE

2.1 Problem Statement

Before the implementation of the EventForce Management System, the organization’s event planning operations relied on highly fragmented, manual processes. Data was siloed across decentralized spreadsheets, physical registers, and

isolated email threads. This lack of a unified digital infrastructure created severe operational bottlenecks and communication gaps across every level of the business.

To architect a highly targeted Salesforce solution, a comprehensive pain-point analysis was conducted across five core stakeholder groups. The distinct challenges faced by each persona highlight the critical need for a centralized CRM platform:

1. Event Operations Manager: The Challenge of Centralized Visibility

- **Core Problem:** The inability to manage event planning information and operations efficiently due to decentralized, scattered data.
- **Operational Impact:** Without a single source of truth, the Operations Manager lacks a holistic, real-time view of the business. Tracking overall business health, event statuses, and budget allocations requires manually aggregating data from different departments. This fragmentation obscures key performance metrics, delays strategic decision-making, and creates a reactive rather than proactive management environment. The absence of centralized reporting means that identifying bottlenecks or calculating total revenue pipeline is a labor-intensive, error-prone process.

2. Event Coordinator: The Burden of Manual Administration

- **Core Problem:** Inefficiencies in creating, modifying, and managing event bookings throughout their lifecycle.
- **Operational Impact:** Event Coordinators are bogged down by redundant administrative tasks. Creating a single event profile, associating it with the correct client, manually updating statuses from "Planned" to "Confirmed," and tracking budgets requires cross-referencing multiple documents. Without automated workflows or standardized digital forms, coordinators are highly susceptible to human error. The excessive manual overhead takes time away from actual event planning and client relationship building, drastically limiting the number of events a single coordinator can successfully manage at one time.

3. Venue Manager: The Risk of Logistical Conflicts

- **Core Problem:** Inaccurate venue availability tracking, leading to the high-risk scenario where the same venue is assigned to multiple events simultaneously.
- **Operational Impact:** Managing physical locations requires absolute scheduling precision. Relying on manual calendars creates a significant blind

spot regarding venue availability and capacity limits. Without programmatic safeguards enforcing schedule integrity, a venue manager can easily double-book a popular location on a high-demand date. These scheduling collisions threaten severe logistical failures, financial penalties, and catastrophic damage to the company's reputation.

4. Vendor Manager: The Complexity of Multi-Event Coordination

- **Core Problem:** The logistical difficulty of coordinating, assigning, and tracking external vendors across overlapping events.
- **Operational Impact:** Event execution relies heavily on a complex web of third-party service providers, from catering and decor to videography and transportation. A single event requires multiple vendors, and a single vendor may be servicing several different events in a single weekend. Tracking which vendor is booked for which event, verifying their current availability status, and ensuring their specific service types align with the event's needs is chaotic when managed via phone calls and spreadsheets. This disjointed coordination leads to scheduling conflicts, missed service deliveries, and frantic last-minute replacements.

5. Client: The Customer Experience and Communication Gap

- **Core Problem:** A lack of proactive communication, leaving clients anxious and without timely event information or upcoming reminders.
- **Operational Impact:** Clients invest significant financial resources and emotional energy into their events, expecting transparency and reliability. When communication relies entirely on the manual effort of busy coordinators, clients are frequently left in the dark. The lack of standardized, automated updates—such as booking confirmations or pre-event logistical reminders—forces clients to initiate contact to verify details. This not only increases the inbound call volume for the support team but also degrades the overall customer journey, leading to lower post-event satisfaction ratings and reduced referral business.

Synthesis: These isolated challenges converge into a singular business mandate. To scale operations, eliminate logistical errors, and elevate the client experience, the organization requires a unified, automated, and cloud-based architecture capable of connecting data, enforcing business logic, and streamlining cross-departmental collaboration.

2.2 Empathy Map Canvas

To ensure the EventForce Management System achieves high user adoption and effectively solves real-world operational bottlenecks, an Empathy Map Canvas was developed for the system's primary power user: the Event Coordinator. This exercise maps their cognitive load, daily environment, and core friction points, ensuring the Salesforce implementation is tailored to their specific workflow requirements.

User Persona Profile

- **Persona / Role:** Event Coordinator
- **Primary Responsibility:** Full lifecycle event management. The coordinator acts as the operational nucleus of the business, responsible for client intake, booking management, venue reservations, multi-vendor coordination, status updates, and timely client communications.

Think & Feel (Internal State)

- **Aspirations:** Wants to manage events smoothly, deliver flawless execution, and provide a highly reliable, stress-free experience to clients.
- **Anxieties:** Constantly worried that a critical detail will slip through the cracks. Feels overwhelmed by the sheer volume of micro-tasks (e.g., remembering to send 3-day reminders, chasing vendor confirmations).
- **Motivations:** Desires a sense of control and confidence over their event pipeline. Wants to shift their energy from tedious data entry to high-value client relationship building and creative event planning.

See & Hear (Environmental Context)

- **See:** Disorganized, decentralized tracking systems. The coordinator stares at disjointed Excel spreadsheets, scattered email threads, physical sticky notes, and conflicting calendar invites across different platforms.
- **Hear:** Continuous, urgent inquiries from all directions. Clients asking for status updates, vendors requesting logistical clarifications, venue managers warning about tight turnaround times, and internal management demanding pipeline reports.

Say & Do (Observable Behavior)

- **Do:** Engages in extreme context-switching. Spends hours manually cross-referencing availability across different registers, manually typing repetitive confirmation emails, and constantly calling stakeholders to verify details.
- **Say:** Frequently has to tell clients, "Let me check our spreadsheets and get back to you," rather than having answers immediately accessible on screen.

Pain (Friction Points & Hurdles)

- **Fragmented Information:** Client details, event budgets, and vendor assignments live in different silos, making it impossible to view a complete event profile at a glance.
- **Manual Coordination:** The lack of automation forces the coordinator to act as a human router, manually triggering every communication, reminder, and status change, which is highly susceptible to human error.
- **Venue Double-Booking Risks:** Because the legacy system lacks programmatic schedule validation, the coordinator lives with the constant, high-stress risk of accidentally assigning the same venue to multiple clients on the same date.
- **Tracking Complexity:** Monitoring the progression of 20+ overlapping events simultaneously—each requiring different vendors and varying approval stages—is cognitively exhausting and unsustainable without a CRM.

Gain (Desired Outcomes & The Salesforce Solution)

- **Centralized Event Information:** Achieving a "single source of truth." The coordinator gains a unified Salesforce UI where Event, Client, Venue, and Vendor data are intrinsically linked via Master-Detail and Lookup relationships.
- **Automated Processes:** Reclaiming hours of lost productivity by offloading repetitive tasks to the system, specifically through Record-Triggered Flows that handle 3-day client reminders and Approval Processes that govern cancellations.
- **Accurate Venue Availability:** Total peace of mind provided by custom Apex triggers that programmatically block overlapping venue bookings and automatically update availability statuses in real time.
- **Improved Operational Visibility:** Gaining the ability to monitor their entire workload via personalized List Views, Reports, and Dashboards, allowing

them to proactively manage their pipeline rather than reactively putting out fires.

3. REQUIREMENT ANALYSIS PHASE

3.1 Customer Journey Map

The Customer Journey Map outlines the end-to-end lifecycle of an event within the EventForce Management System. By mapping the operational workflow from initial contact to post-event analysis, the organization can precisely align Salesforce's declarative and programmatic tools with real-world business requirements. This expanded journey details the business objectives, user interactions, system architecture, and operational value at each stage of the event lifecycle.

Stage 1: Event Enquiry & Client Registration

Business Objective: The journey begins at the point of initial contact. The primary goal at this stage is to capture accurate, standardized client demographic and contact information, establishing the foundational record upon which all future communications and event logistics will be built.

User Actions: When a prospective client makes an inquiry, the Event Coordinator navigates to the custom "Clients" tab within the Event Planner Lightning App. The coordinator initiates a new record creation, inputting the client's name, email address, phone number, physical address, country, and city.

System Integration (Salesforce):

- **Data Model:** This step utilizes the custom Client__c object.
- **Data Integrity Automation:** To prevent dirty data from corrupting future automated communications, the system enforces a custom Validation Rule named Email_Valid_Address. If the coordinator inputs an improperly formatted email, the system evaluates a REGEX formula and instantly blocks the save, displaying the error message: "Please Enter Valid Email Address".

Operational Value: By enforcing data standards at the point of entry, the organization eliminates downstream communication failures. The client profile

becomes a centralized "single source of truth," ensuring that any coordinator interacting with the client has access to accurate, up-to-date contact details.

Stage 2: Event Planning & Booking

Business Objective: Once the client is registered, the scope, scale, and financial parameters of the requested event must be defined. This stage transitions a preliminary inquiry into a structured, trackable project.

User Actions: The Event Coordinator creates a new event record from the "Events" tab. They define core parameters by selecting the Event Type (e.g., Wedding, Corporate, Concert) and Event Status (initially set to "Planned"), and select the specific Event Date. The coordinator then links this event to the previously created Client record.

System Integration (Salesforce):

- **Data Model:** This utilizes the central Event__c custom object. The link to the client is established via a Lookup Relationship (Client__c on the Event object).
- **Financial Automation:** The system automatically calculates the financial baseline using a Custom Formula Field (Event_Budget__c). Using a CASE function, the system evaluates the selected Event Type and instantly generates the appropriate budget (e.g., automatically populating $50,000 for a Wedding or $30,000 for a Corporate event).

Operational Value: Automating the budget calculation standardizes pricing across the organization and eliminates manual arithmetic errors. Linking the Event to the Client establishes the relational database structure, allowing management to view a client's entire booking history from a single screen.

Stage 3: Venue Selection & Reservation

Business Objective: Securing the physical location for the event is a high-risk operational phase. The objective is to identify a suitable venue, verify its capacity, and officially reserve it for the specific event date while guaranteeing zero scheduling conflicts.

User Actions: The coordinator reviews available venues and assigns the chosen location to the event by updating the Venue lookup field on the Event record. Once logistics are finalized, the coordinator changes the Event Status to "Confirmed".

System Integration (Salesforce):

- **Data Model:** This step connects the Event__c object to the Venue__c object via a Lookup Relationship.
- **Programmatic Validation (Apex Trigger):** To eliminate the catastrophic risk of overlapping bookings, an Apex Trigger named PreventDoubleBooking fires before insert and before update. The system queries the database to check if the selected Venue is already linked to an Event on the same date. If a conflict is detected, the trigger blocks the transaction and throws a real-time .addError() message to the user.
- **Status Automation (Apex Class):** Once confirmed, the VenueStatusHelper Apex class automatically updates the linked Venue's "Availability Status" field to "Reserved".

Operational Value: This stage replaces manual calendar checking with strict, programmatic governance. By letting Apex logic handle conflict resolution, the organization completely neutralizes the risk of double-booking, protecting both revenue and brand reputation.

Stage 4: Vendor Coordination

Business Objective: Modern events require complex supply chains. This stage focuses on assembling and managing the network of third-party service providers (e.g., Catering, Decor, Photography, Lighting) required to execute the event.

User Actions: The Vendor Manager reviews the event requirements and navigates to the "Event Vendors" related list on the Event record. They click "New" to assign multiple external vendors to the single event, ensuring all required service types are covered.

System Integration (Salesforce):

- **Data Model:** Because an Event requires multiple Vendors, and a Vendor can service multiple Events, Salesforce requires a Many-to-Many relationship. This is handled by the EventVendor__c Junction Object.

- **Relationship Architecture:** The Junction Object contains two Master-Detail relationships: one pointing to Event__c and one pointing to Vendor__c.

Operational Value: The junction object architecture breaks down operational silos. Vendor Managers can view a Vendor's record and see every event they are assigned to, while Event Coordinators can look at an Event and instantly see the entire roster of hired vendors. This centralized visibility prevents scheduling gaps and missed service deliveries.

Stage 5: Event Confirmation & Communication

Business Objective: Consistent, proactive communication is critical for client peace of mind. This stage ensures that once logistics are locked in, the client is automatically kept in the loop without requiring manual effort from the coordination team.

User Actions: The user action here is minimal by design. The coordinator simply ensures the Event Status is saved as "Confirmed". The system handles the rest.

System Integration (Salesforce):

- **Automation Engine (Flows):** The system utilizes a Record-Triggered Flow optimized for "Actions and Related Records".
- **Time-Based Execution:** The Flow features a Scheduled Path named 3_Days_Before, which uses the Event_Date__c as its time source and applies an offset of 3 days prior.
- **Dynamic Messaging:** When the date arrives, the Flow triggers an Email Action (Alert_Client_3Day_Reminder), pulling dynamic Text Templates that inject the Client's Name, Event Name, Date, and Venue directly into the email body.

Operational Value: By automating pre-event reminders, the organization drastically reduces inbound support calls from anxious clients. It guarantees a standardized, professional touchpoint for every single booking, elevating the overall customer experience while freeing up the coordinator's time.

Stage 6: Event Execution & Monitoring

Business Objective: As events approach and execute, organizational leadership requires a macro-level view of operations to monitor bandwidth, forecast revenue, and track overall business health.

User Actions: Event Operations Managers and executives navigate to the "Dashboards" and "Reports" tabs within the Lightning App. They interact with visual charts to drill down into specific months or event types.

System Integration (Salesforce):

- **Analytics Architecture:** The system relies on a custom Report named "Upcoming Events — Summary by Month", which groups event records by their Calendar Month and calculates the sum of the Event Budgets.
- **Data Visualization:** This report feeds directly into the "EventForce Operations Dashboard", displaying a Donut Chart that visualizes upcoming event volume and financial pipeline in real time.

Operational Value: Data visualization transforms raw CRM records into actionable business intelligence. Management can instantly identify peak seasons, anticipate resource bottlenecks, and make strategic decisions based on live, accurate data rather than outdated spreadsheets.

Stage 7: Cancellation & Approval

Business Objective: Event cancellations carry significant financial and logistical consequences (e.g., releasing venues, canceling vendor contracts). This stage enforces strict administrative governance, ensuring no event is canceled without proper managerial review.

User Actions: If a client requests a cancellation, the Event Coordinator cannot delete the record. Instead, they update the Event Status to "Pending Cancellation" and click the "Submit for Approval" button.

System Integration (Salesforce):

- **Governance Automation:** This triggers the Event_Cancellation_Process Approval Process.
- **Routing & Alerts:** The record is locked, and an Initial Submission Email Alert is routed to the designated Manager.
- **Final Actions:** If the manager approves, Final Approval Actions automatically trigger a Field Update (changing the status definitively to "Canceled") and dispatch automated Email Alerts notifying both the Event Owner and the Client. If rejected, the status reverts, and the owner is notified to proceed with the event.

Operational Value: This standardized workflow prevents rogue cancellations and ensures that management has the opportunity to review contracts or attempt to save the account before logistical teardown begins. It also guarantees that all stakeholders (clients, owners, and eventually vendors) are notified uniformly.

Stage 8: Feedback & Closure

Business Objective: The final stage closes the operational loop. The objective is to automatically archive completed events to keep the active pipeline clean, while systematically capturing client satisfaction data for continuous business improvement.

User Actions: Following the event, the coordinator or client inputs a post-event rating (1-5) and text comments into a new Feedback record.

System Integration (Salesforce):

- **Data Model & UI Control:** The Feedback__c object links to both the Client and the Event. To ensure data accuracy, a Lookup Filter is applied to the Event field on the Feedback record: it restricts the user to only selecting events that belong to the chosen Client, preventing cross-client data contamination.
- **Asynchronous Automation (Batch Apex):** To close out the records, a nightly Batch Apex job (BatchCompleteEvents) runs via a Schedulable class (ScheduleCompleteEvents). Every night at 8:00 PM, the system queries all events where the Event Date is in the past, and automatically updates their Event Status to "Completed".

Operational Value: Automated closure ensures the active CRM pipeline is never cluttered with past events, maintaining system performance and reporting accuracy. The structured capture of feedback allows the business to measure vendor performance, client satisfaction, and operational success, providing the insights needed to refine future event execution.

4. PROJECT DESIGN PHASE

The EventForce Management System is engineered on the Salesforce platform, utilizing a Model-View-Controller (MVC) architectural pattern. The system design strictly adheres to Salesforce best practices, prioritizing declarative (clicks, not code) configurations for standard automation, while **4.1 Solution Architecture**

strategically deploying programmatic (Apex) solutions for complex, cross-object transactional logic. The architecture is divided into five distinct layers: the Data Model Layer, the Declarative Logic Layer, the Programmatic Logic Layer, the Security Layer, and the Presentation/Analytics Layer.

1. Data Model Layer (Custom Objects & Relationships)

The foundation of the EventForce architecture is its robust, highly relational custom data model. Standard Salesforce objects were bypassed in favor of a fully customized schema to perfectly map to the event management lifecycle.

- **Core Entities:** The system revolves around the Event__c object, which acts as the central hub tracking dates, budgets, and statuses. Peripheral entities include Client__c (customer demographics), Venue__c (location capacity and availability), Vendor__c (service providers), and Feedback__c (post-event satisfaction metrics).
- **Relational Architecture:** These objects are connected via strict relationship fields to ensure data consistency. Event__c has Lookup relationships to both Client__c and Venue__c. Feedback__c looks up to both the Client and the Event, utilizing a Lookup Filter to ensure a client can only leave feedback for their own events.
- **The Junction Object:** Because a single event requires multiple vendors (e.g., catering, lighting) and a single vendor services multiple events, the architecture employs an EventVendor__c junction object. By using two

Master-Detail relationships pointing to Event and Vendor, this architecture seamlessly resolves the Many-to-Many data requirement.

2. Declarative Logic & Automation Layer

To maintain a scalable and easily maintainable system, EventForce utilizes Salesforce's native declarative tools to enforce data integrity and automate standard business processes.

- **Validation Rules & Formula Fields:** Data hygiene is enforced at the database level. For example, the Email_Valid_Address Validation Rule uses a REGEX formula to ensure malformed contact data never enters the system. Simultaneously, Formula Fields offload manual calculations to the server; the Event_Budget__c field dynamically calculates expected costs based on the selected Event Type.
- **Record-Triggered Flows:** Time-based communication is automated using Salesforce Flows. The architecture includes a Flow with a Scheduled Path that evaluates when an event status is "Confirmed" and automatically dispatches a personalized reminder email exactly three days prior to the Event_Date__c. This creates a reliable, asynchronous background process without writing code.
- **Approval Processes:** Administrative governance over event cancellations is architected via a multi-step Approval Process. This component locks the event record when the status enters "Pending Cancellation" and routes the request to a defined Manager. It orchestrates automated field updates (changing status to "Canceled" or "Rejected") and dispatches targeted email alerts based on the manager's decision.

3. Programmatic Logic Layer (Apex Integration)

When business requirements exceed the limitations of declarative tools—specifically regarding complex cross-object querying and bulk data manipulation—the architecture relies on custom Apex code.

- **Apex Triggers & Classes (Real-Time Execution):** To maintain strict logistical integrity, the PreventDoubleBooking Apex Trigger executes before insert and before update. It queries the database for existing events linked to the selected Venue and throws an .addError() exception if a date conflict is detected. Additionally, the VenueStatusHelper Apex Class programmatically

synchronizes the Venue's availability status based on the linked Event's lifecycle.

- **Batch & Scheduled Apex (Asynchronous Execution):** To maintain system hygiene without manual intervention, the architecture incorporates asynchronous processing. The BatchCompleteEvents class implements the Database.Batchable interface to safely process large volumes of records without hitting governor limits. This is paired with the ScheduleCompleteEvents class, which acts as a cron job, executing the batch process every night at 8:00 PM to automatically transition past events to a "Completed" status.

4. Security & Access Control Layer

The EventForce architecture employs a "Principle of Least Privilege" security model, ensuring users only access the data required for their specific job function.

- **Profiles & Roles:** The system uses custom Profiles (Event Admin, Event Coordinator, Vendor Manager, Client Profile) to define baseline Object and Field-level permissions (e.g., Create, Read, Edit, Delete). The Role Hierarchy (CEO -> Event Admin -> Coordinator/Vendor Manager) dictates the vertical sharing of records.
- **Organization-Wide Defaults (OWD) & Sharing Rules:** To protect sensitive client data, the OWD for the Event object is set to strictly "Private". Horizontal record sharing is architected via Criteria-Based Sharing Rules, specifically the Event_Sharing_For_Vendors rule, which selectively grants Read-Only access to Vendor Managers for specific event records.
- **Permission Sets:** Exception-based access is handled via Permission Sets. The Feedback Manager permission set grants specific administrative rights over the Feedback object without requiring a completely separate Profile for the user.

5. Presentation & Analytics Layer (UI/UX)

The top layer of the architecture focuses on user experience, data visualization, and operational efficiency.

- **Lightning App:** All custom objects, tabs, and analytics are consolidated into the "Event Planner" Lightning App. This provides a unified, branded, and intuitive graphical user interface (GUI) for the end-users, drastically reducing the time spent navigating between disparate tools.
- **Reports & Dashboards:** The architecture leverages Salesforce's native analytics engine. Data is aggregated via customized reports, such as the "Upcoming Events — Summary by Month" report, which groups records by date and sums the budget fields. This tabular data feeds into the visual "EventForce Operations Dashboard," utilizing dynamic Donut Charts to give leadership real-time, actionable insights into event volume and financial pipelines.

5. PROJECT PLANNING & SCHEDULING

To ensure the successful delivery of the EventForce Management System within the designated timeframe, the project was executed using a highly structured project management framework. Given the dynamic nature of CRM development and the need for continuous stakeholder feedback, traditional waterfall methodologies were discarded in favor of an iterative, agile approach. This section outlines the project management lifecycle, detailing how high-level requirements were deconstructed into actionable technical tasks and delivered through synchronized sprints.

5.1 Agile Methodology Adoption: The EventForce project was governed by the Agile methodology. Agile was selected because it aligns perfectly with the rapid configuration capabilities of the Salesforce platform. Instead of attempting a single, massive deployment at the end of the project timeline, Agile allowed the development team to deliver functional increments of the CRM continuously.

This iterative approach provided several core operational advantages:

- **Continuous Feedback Loops:** By demonstrating functional Salesforce components (such as a working Flow or a new custom object) at the end of short cycles, stakeholders could provide immediate feedback, ensuring the final system accurately reflected business needs.
- **Adaptability:** Event management requirements often shift. Agile allowed the development team to pivot easily, adjusting data models or validation rules without derailing the entire project timeline.

- **Risk Mitigation:** By tackling the most complex programmatic elements (such as Apex triggers for double-booking prevention) in isolated phases, technical roadblocks were identified and resolved early.

5.2 Requirements Structuring: Epics to Story Points To manage the workload efficiently, all business requirements were translated into a hierarchical Agile structure consisting of Epics, User Stories, and Story Points.

- **Epics:** These represented large, overarching business requirements or functional modules. For the EventForce system, core Epics included "Event & Client Management," "Vendor Coordination," "Automated Notifications," "Approval Workflows," and "Analytics & Security".
- **User Stories:** Each Epic was broken down into granular User Stories. These stories were written from the perspective of the end-user to ensure that every technical configuration directly solved a human problem. The format strictly followed the standard Agile syntax: *"As a [Role], I want to [Action] so that [Benefit]."*
- **Story Points & Estimation:** Rather than estimating tasks in hours—which fails to account for technical unknowns—the team utilized Story Points. Story points calculate the relative effort, complexity, and uncertainty of a task using a Fibonacci-like sequence (e.g., 3, 5, 8). For example, configuring a basic standard object might be assigned a 3, whereas writing and testing a complex Apex trigger to query overlapping venue dates was assigned an 8 due to its high technical complexity.
- **Velocity Calculation:** Velocity calculation was utilized to measure the total number of story points the team could successfully burn through during a single sprint. This metric was crucial for accurate capacity planning and determining the final project timeline.

5.3 Sprint-Based Execution Strategy and Product Backlog: The actual build of the Salesforce org was conducted through Sprint-based execution. A Sprint is a time-boxed iteration during which a specific set of user stories is developed, tested, and deployed. For the EventForce project, the team executed four distinct sprints, each with a strict duration of 6 days and a target velocity of 20 story points.

The Product Backlog and Sprint Schedule were mapped out as follows to ensure logical system progression:

Sprint 1: Core Data Architecture (Oct 24, 2022 – Oct 29, 2022)

The first sprint was dedicated to establishing the foundational database schema. Without these custom objects, no automation or relationships could exist.

- **US-01 (Event Management Epic):** *"As an Event Coordinator, I want to create events with type, date, and status so that event information is standardized."* This 3-point, high-priority story involved creating the central Event__c object and its core picklist fields. It was assigned to Team Member 1.
- **US-02 (Client & Venue Management Epic):** *"As an Event Coordinator, I want to associate a client and venue with an event so that coordination is centralized."* This 5-point, high-priority story required the creation of the Client__c and Venue__c objects, alongside the Lookup relationships linking them to the Event. It was assigned to Team Member 2.

Sprint 2: Ecosystem Expansion & Automation (Oct 31, 2022 – Nov 05, 2022)

With the core objects built, the second sprint focused on expanding the architecture to include third-party vendors and introducing declarative automation.

- **US-03 (Vendor Management Epic):** *"As a Vendor Manager, I want to assign multiple vendors to an event so that services can be coordinated."* This 5-point, high-priority story was technically demanding, requiring the creation of the Vendor__c object and the EventVendor__c Many-to-Many junction object. It was assigned to Team Member 3.
- **US-04 (Notifications & Automation Epic):** *"As a Client, I want timely event reminders so that I know when my event is approaching."* This 3-point, high-priority story involved building the Record-Triggered Flow with a Scheduled Path to automate the 3-day reminder emails. It was assigned to Team Member 4.

Sprint 3: Advanced Programmatic Logic (Nov 07, 2022 – Nov 12, 2022)

Sprint 3 was the most technically intensive phase, shifting focus from declarative "clicks" to programmatic "code" and structured governance.

- **US-05 (Approval Management Epic):** *"As an Event Coordinator, I want cancellation requests to follow an approval process so that cancellations are controlled."* This 5-point, high-priority story required mapping out the multi-

step Event_Cancellation_Process with email templates and field updates. It was assigned to Team Member 1.

- **US-06 (Venue Management & Apex Epic):** *"As a Venue Manager, I want double bookings prevented so that venue schedules remain accurate."* As the most complex task in the backlog, this was weighted at 8 story points. It required writing, testing, and deploying the PreventDoubleBooking Apex trigger to query and block scheduling conflicts. It was assigned to Team Member 2.

Sprint 4: Security, Analytics, and Finalization (Nov 14, 2022 – Nov 19, 2022)

The final sprint focused on locking down the system architecture and providing visual analytics to management prior to deployment.

- **US-07 (Security & Access Control Epic):** *"As an Event Administrator, I want role-based access so that users see only appropriate information."* This 5-point, high-priority story involved configuring the entire security model: Profiles, Roles, Private Organization-Wide Defaults (OWD), and Sharing Rules. It was assigned to Team Member 3.
- **US-08 (Reports & Dashboards Epic):** *"As Management, I want reports and dashboards so that event operations can be monitored."* This 5-point, medium-priority story wrapped up the build by creating the "Upcoming Events by Month" report and visualizing it on the EventForce Operations Dashboard. It was assigned to Team Member 4.

5.4 Project Tracker, Velocity, and Burndown Management: To guarantee the project remained on schedule, the Scrum Master maintained a rigorous Project Tracker, Velocity Chart, and Burndown Chart. The Burndown Chart provided a daily visual representation of remaining work versus time, highlighting instantly if a sprint was at risk of failure.

The team established a highly consistent cadence, committing to and delivering exactly 20 story points per 6-day sprint cycle.

- **Sprint 1:** Started exactly as planned on October 24, 2022. All 20 points were completed, and the sprint was released on time on October 29, 2022.
- **Sprint 2:** Commenced on October 31, 2022, tackling 20 points, and concluding on November 05, 2022.

- **Sprint 3:** Commenced on November 07, 2022, tackling 20 points (including the heavy 8-point Apex story), and concluding on November 12, 2022.
- **Sprint 4:** Commenced on November 14, 2022, tackling the final 20 points, and closing out the development lifecycle exactly on schedule on November 19, 2022.

By strictly adhering to Agile velocity calculations and sprint-based execution, the development team avoided scope creep, managed technical debt effectively, and delivered a fully functional, enterprise-grade Salesforce CRM reliably within a single month of development time.

6. PROJECT DEVELOPMENT PHASE :

PHASE 1 : REQUIREMENT ANALYSIS & STRATEGIC PLANNING

1. Project Vision & Business Case The EventForce Management System is conceived to resolve the operational inefficiencies inherent in fragmented event planning processes. Currently, managing client bookings, coordinating third-party vendors, reserving physical venues, and tracking customer satisfaction requires navigating multiple disconnected platforms or spreadsheets. This project will deploy a centralized, highly relational Salesforce CRM architecture to unify these workflows. By automating routine scheduling, enforcing approval governance, and providing real-time data visibility, EventForce will empower the operations team to scale event capacity while delivering a flawless, data-driven customer experience.

2. Functional Scope & System Architecture To achieve the project vision, the system will be built upon a robust foundation of Salesforce declarative and programmatic tools. The core functional requirements are categorized as follows:

- **Relational Data Modeling:** The architecture relies on custom objects (Event__c, Client__c, Vendor__c, Venue__c, and Feedback__c). Crucially, it will utilize a Junction Object (Event_Vendor__c) to establish a Many-to-Many relationship, allowing a single event to utilize multiple vendors, and a single vendor to service multiple events simultaneously. Master-Detail and Lookup relationships will bind the remaining objects.

- **Declarative Automation & Workflow:** Record-Triggered Flows will handle time-based actions, such as dispatching 3-day client reminders and triggering automated feedback collection post-event.
- **Governance & Approvals:** Standardizing exception management through formalized Approval Processes. Event cancellations will no longer be unilateral; they will route to management for sign-off, automatically triggering status updates and email alerts to the assigned coordinator and the client.
- **Data Integrity & Financial Logic:** Deployment of strict Validation Rules to ensure accurate data entry (e.g., preventing past dates for new events) and Formula Fields to dynamically calculate overall event budgets and vendor cost roll-ups.
- **Programmatic Validation (Apex):** Where declarative tools reach their limits, Apex Classes and Triggers will be engineered to actively query database records, automatically updating venue availability and acting as a hard stop to prevent overlapping double-bookings at the same location.
- **Security & Access Control:** Implementation of a strict security matrix utilizing custom Profiles, a vertical Role Hierarchy, and modular Permission Sets to enforce the Principle of Least Privilege across the organization.
- **Business Intelligence:** A suite of custom Reports and real-time Dashboards to provide executive visibility into upcoming event volume, vendor performance ratings, and overall operational health.

3. Stakeholder Persona Mapping Understanding the end-users is critical to system adoption. The architecture is designed to serve five distinct stakeholder groups:

- **Clients (End-Users):** Seek transparency and reliability. They interact with the system via automated communications, submitting initial service requests, receiving timely event updates, and providing post-event satisfaction feedback.
- **Event Coordinators (Primary Operators):** The heaviest users of the system. They require a unified workspace to manage scheduling, log client communications, assign vendors, and initiate cancellation approval workflows without digital friction.
- **Vendors (External Partners):** Third-party service providers (catering, A/V, decor, logistics). They require clear, documented assignments linked directly to specific events, ensuring they understand their deliverables and schedules.
- **Venue Managers (Resource Controllers):** Responsible for the physical locations. They rely on the system's Apex-driven automation to ensure venue

availability statuses are strictly maintained and immune to scheduling conflicts.

- **Event Administrators (System Owners):** Responsible for platform health. They require deep visibility into system operations, managing the security architecture (Profiles/Roles), and monitoring high-level data trends via executive dashboards.

4. Execution Roadmap (Phased Delivery) The implementation of the EventForce Management System will follow a structured, sequential methodology to ensure foundational stability before introducing complex automation:

- **Step 1: Foundational Architecture (Data Model):** Creation of the custom objects (Event, Client, Vendor, Venue, Feedback), custom fields, and the critical relational web (Lookups, Master-Details, and the Event Vendor junction object).
- **Step 2: Data Integrity & UI Configuration:** Implementation of Validation Rules to enforce data quality and Formula Fields for financial calculations. Layout optimization and Lightning App creation.
- **Step 3: Declarative Automation:** Engineering Record-Triggered Flows to automate routine tasks, such as client reminders and status updates.
- **Step 4: Approval Governance:** Building the multi-step Approval Process to manage event cancellation requests, including the configuration of email alerts and automated field updates upon rejection or approval.
- **Step 5: Programmatic Expansion:** Developing and testing Apex Classes and Triggers to handle complex cross-object logic, specifically managing real-time venue availability and double-booking prevention.
- **Step 6: Security Matrix Implementation:** Configuring the Organization-Wide Defaults (OWD), cloning custom Profiles, establishing the Role Hierarchy, and deploying Permission Sets to lock down record visibility.
- **Step 7: Analytics & Deployment:** Building the operational Reports and executive Dashboards, followed by final data migration, user acceptance testing (UAT), and system go-live.

PHASE 2 : BACKEND DEVELOPMENT & CONFIGURATIONS

The Backend & Configuration phase serves as the central nervous system of the EventForce Management System. Transitioning the platform from a static database into an intelligent, automated operational engine, this phase focused on bridging declarative tools and programmatic code to enforce business logic, eliminate manual administrative friction, and secure enterprise data.

Phase 1: Declarative Automation & Workflow Governance (Clicks over Code) To maximize system maintainability and adhere to Salesforce best practices, the majority of standard business processes were automated using declarative (no-code) tools.

- **Data Integrity & Financial Roll-ups:** Validation Rules act as the first line of defense, intercepting bad data at the UI level (e.g., preventing coordinators from scheduling events in the past). Concurrently, custom Formula Fields and Roll-Up Summaries dynamically calculate and aggregate vendor costs, providing real-time budget tracking without requiring manual spreadsheet calculations.
- **Time-Based Communication (Flows):** Record-Triggered Flows were engineered to handle the system's communication layer. By utilizing Scheduled Paths, the system automatically queues and dispatches standardized 3-day reminder emails to clients and post-event feedback requests, ensuring consistent customer engagement while freeing coordinators from manual tracking.
- **Exception Management (Approval Processes):** To enforce operational governance, unilateral event cancellations were disabled. Instead, cancellation requests are routed through a formalized multi-step Approval Process. This mechanism temporarily locks the record, escalates the request to the Event Admin, and automatically executes field updates and email alerts based on the manager's final approval or rejection.

Phase 2: Programmatic Business Logic (Apex) When complex, cross-object transactional requirements exceeded the limitations of declarative tools, custom programmatic logic was deployed using Apex, Salesforce's object-oriented programming language.

- **Real-Time Conflict Prevention:** The PreventDoubleBooking Apex Trigger was developed to actively intercept database insertions and updates. By querying the entire database for existing reservations, the trigger instantly

blocks overlapping schedules at the same venue, throwing a custom UI error and neutralizing logistical disasters before they occur.

- **Status Synchronization:** The VenueStatusHelper Apex Class programmatically binds Event statuses to physical Venue records. When an event is confirmed or canceled, the Apex logic dynamically updates the linked venue's availability status (Reserved vs. Available) in real-time.
- **Asynchronous Processing:** To maintain system hygiene without manual intervention, Schedulable Batch Apex was implemented. Acting as a nightly cron job, this script sweeps the database during off-peak hours, identifying past events and automatically advancing their status to 'Completed'.

Phase 3: Multi-Layered Security & Access Control A robust CRM is inherently vulnerable without a strict data governance model. This phase implemented a "Zero Trust" security architecture designed to enforce the Principle of Least Privilege across all organizational tiers.

- **Object-Level Security:** Standard profiles were bypassed in favor of highly customized, cloned Profiles (Event Coordinator, Vendor Manager, Client), dictating the exact Create, Read, Edit, and Delete (CRED) baseline for every object.
- **Record-Level Visibility:** The Organization-Wide Defaults (OWD) were locked down to 'Private'. Visibility was then strategically reopened using a vertical Role Hierarchy (allowing management to inherit access to subordinate data) and lateral, criteria-based Sharing Rules (securely exposing specific Coordinator records to Vendor Managers as 'Read Only').
- **Modular Permissions:** To prevent profile bloat, exception-based access—such as granting specific senior coordinators the rights to manage and delete customer feedback—was handled exclusively through modular Permission Sets.

By synchronizing declarative workflows, robust programmatic safeguards, and a multi-layered security matrix, this phase successfully transformed EventForce into a highly scalable, secure, and fully automated enterprise ecosystem.

Milestone 1: Salesforce Account Setup

Activity 1: Creating Developer Account

1. Objective To provision a new, isolated Salesforce environment (Developer Edition) exclusively for building the EventForce Management System. This environment serves as the foundational sandbox where all subsequent metadata, custom objects, relationships, declarative automations, and programmatic code will be built, tested, and validated without risking production data integrity.

2. Theoretical Background & Context A Salesforce Developer Edition (DE) organization is a free, fully-featured copy of the Enterprise Edition environment. It provides a safe, independent sandbox for developers, administrators, and architects to create custom objects, write Apex code, configure record-triggered flows, and establish security hierarchies. Setting this up is the mandatory, foundational step for the entire project lifecycle.

In enterprise environments, direct configuration or coding in production is heavily restricted due to the risks of data corruption and service interruption. Developer Edition orgs replicate enterprise-grade metadata deployment structures, allowing developers to test features thoroughly before moving them across environments via change sets or continuous integration pipelines. Furthermore, because the EventForce project requires advanced features like custom triggers, batchable asynchronous Apex, and custom object relationships, provisioning a dedicated Developer Edition org ensures that the team has full administrative ("System Administrator") privileges, unrestricted access to the Salesforce Setup menu, and a clean slate free of pre-existing legacy data constraints.

3. Step-by-Step Execution Guide

1. Open a secure web browser and navigate to the official Salesforce developer sign-up portal at https://developer.salesforce.com/signup.
2. On the registration landing page, locate the sign-up form and carefully enter your legal First Name and Last Name in the designated text input fields.
3. Provide a valid, actively monitored, and accessible Email address in the email field; this is crucial for receiving account verification links, system notifications, and password reset tokens.

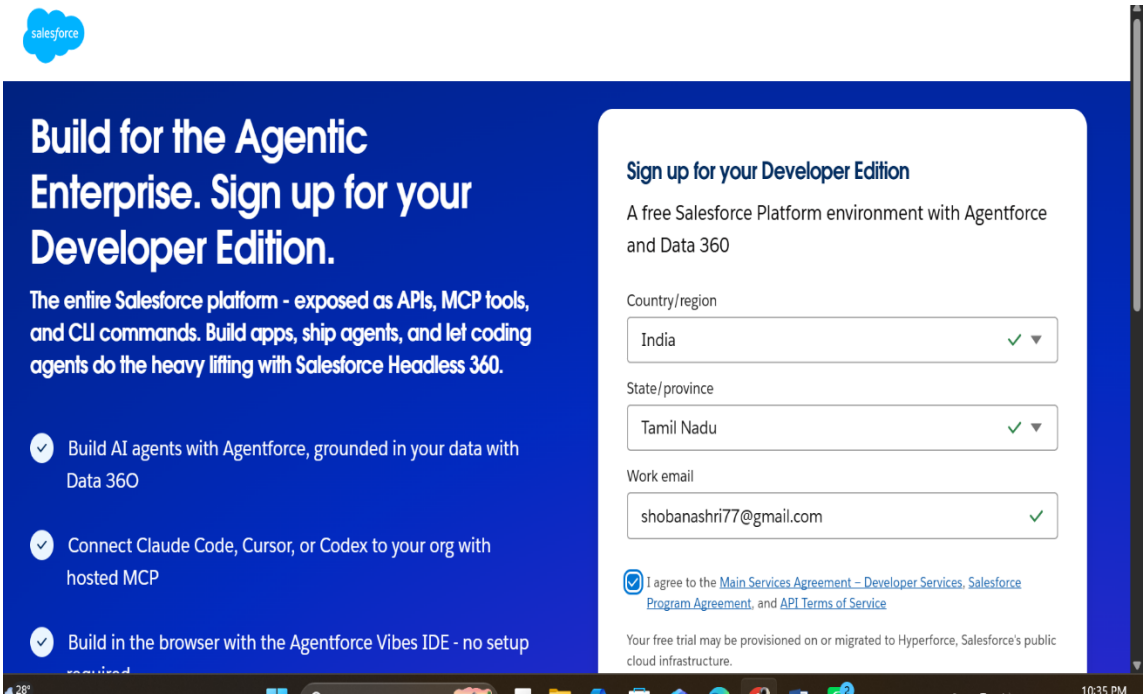
4. Locate the Role dropdown menu on the form and select 'Developer' to configure the profile type appropriately for software engineering and CRM configuration tasks.
5. Enter your academic institution's name or your organization name in the Company field (e.g., entering your college name as specified in project parameters).
6. Select your geographical region, choosing 'India' (or your respective country of residence) from the Country selection dropdown.
7. Enter your local postal code or pin code accurately in the Postal Code field to complete the regional address mapping.
8. Create a globally unique Username formatted as an email address (e.g., shobana.eventforce@organization.com). *Note:* This string does not need to be an active, functioning email inbox, but it must be entirely unique across the entire global Salesforce multi-tenant cloud architecture to prevent collision errors.
9. Review all entered data fields for accuracy, check any required terms-of-service compliance boxes, and click the 'Sign me up' button to submit the registration request to Salesforce's provisioning servers.

4. Verification & Testing To verify that the account provisioning request was successfully processed by Salesforce, check the inbox of the primary email address provided during registration. You should receive an automated notification titled 'Welcome to Salesforce: Verify your account' typically within 5 to 10 minutes of form submission. Receipt of this email confirms that your unique username has been successfully registered in the Salesforce cloud infrastructure and is awaiting activation.

5. Best Practices & Troubleshooting

- Always construct your username using a distinctive, multi-part string combination (such as combining your name, project identifier, and a custom domain extension) to completely avoid common 'username already exists' provisioning errors.
- Record and keep your provisioned login credentials (Username, Password, and Security Question answers) saved in a secure password manager or encrypted document, as this exact Developer Edition environment will be utilized heavily across all subsequent phases of the project.

- **Troubleshooting:** If the activation email does not arrive within the expected 10-minute window, immediately check your email provider's Spam, Junk, or Promotions folders. If the email is missing entirely, return to the sign-up page and attempt the registration process again, ensuring there are no hidden typos or syntax errors in your designated email address.



Activity 2: Account Activation

1. Objective

To verify the registered email address, configure a cryptographically secure administrative password, establish foundational identity verification controls, and

gain initial administrative access to the Salesforce Setup backend for the EventForce Management System.

2. Theoretical Background & Context

Enterprise cloud platforms like Salesforce maintain a strict identity verification protocol to ensure that organizational data remains protected from unauthorized access right from inception. The account activation process serves as the critical bridge between cloud provisioning and administrative ownership. By completing this workflow, the system binds the user's provisioned email address and unique username to a verified administrative user record within the multi-tenant cloud infrastructure.

Furthermore, this stage establishes the initial security posture of the Developer Edition organization by enforcing robust password complexity standards and configuring a secret security question. In enterprise Salesforce deployments, administrative accounts hold elevated privileges—such as the ability to modify system-wide settings, deploy Apex triggers, write custom automation flows, and manage data sharing models. Consequently, securing this initial access point prevents unauthorized credential compromise and aligns with foundational cybersecurity compliance frameworks.

3. Step-by-Step Execution Guide

1. Open the primary web mail client or inbox associated with the email address submitted during the Salesforce Developer Edition registration phase.
2. Locate the automated correspondence dispatched from Salesforce, typically subject-lined *"Welcome to Salesforce: Verify your account"*.
3. Open the email message and click the embedded, secure hyperlinked button labeled **'Verify Account'**. This action will launch a new secure browser tab redirecting you directly to the official Salesforce identity verification and password establishment portal.
4. Review the password policy requirements displayed on-screen, and enter a strong, enterprise-grade password into the initial password field. The password must strictly comply with Salesforce's default security baseline—requiring a minimum length of 8 characters and incorporating a secure alphanumeric combination of uppercase letters, lowercase letters, and numbers.
5. Re-enter the exact password string into the confirmation input box to verify character accuracy and prevent typographical errors.

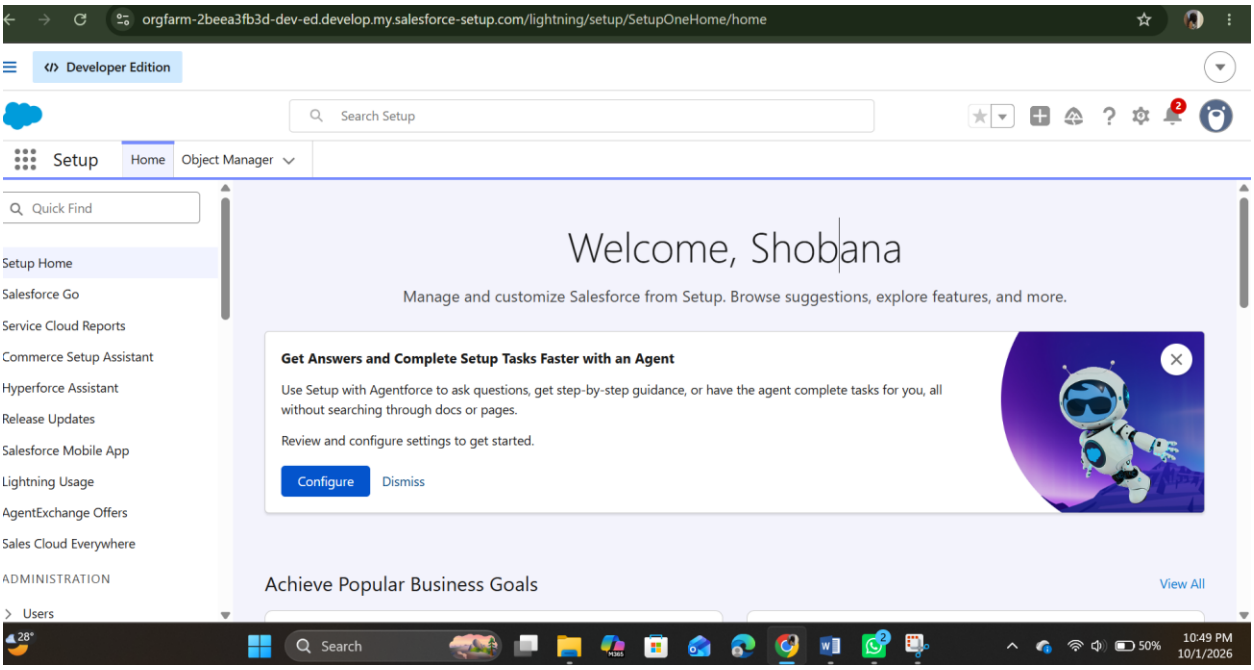
6. Locate the security question selection dropdown menu and choose a preferred challenge question from the provided options (e.g., *"In what city were you born?"* or *"What is your maternal grandmother's maiden name?"*).
7. Enter a clear, memorable, and accurate response into the corresponding answer text field to ensure account recovery options are securely established.
8. Click the **'Change Password'** button to commit your credentials to the cloud directory service and finalize the identity activation sequence.
9. Upon successful database validation of your credentials, the system will automatically log your session in and redirect you to the core Salesforce Setup Home page, initializing your administrative workspace.

4. Verification & Testing To verify that the account activation and security configuration were completed successfully, inspect the browser interface. Upon redirection, you should land directly within the Salesforce Lightning Experience user interface. Specifically, confirm that the top navigation bar displays the core administrative control elements—such as the gear icon (Setup menu), the global search bar, and the Object Manager tab. Accessing the Setup menu without encountering authentication errors or permission blocks confirms that your user record holds full administrative privileges within the newly provisioned organization.

5. Best Practices & Troubleshooting

- **URL Bookmarking:** Always bookmark the standard enterprise login portal (https://login.salesforce.com) in your primary web browser to ensure swift, secure access for all future development sessions.
- **Credential Hygiene:** Never misplace or forget your chosen security question answer and password combination. In free Developer Edition environments, administrative password resets are straightforward via email, but losing access to both the secondary verification mechanisms and the inbox can permanently lock you out of your custom configuration metadata.
- **Browser Configuration:** Ensure that your web browser is configured to allow pop-ups and redirects from Salesforce domains. Many advanced setup menus, developer consoles, and authorization dialogs launch in independent browser windows or utility tabs that can be aggressively blocked by strict ad-blockers or default browser security policies.
- **Troubleshooting Delayed Emails:** If the activation link takes longer than 10 minutes to arrive, verify that your network connection is stable and check your

email provider's spam or junk filters. If necessary, re-initiate the registration sequence to trigger a fresh security token generation.



Milestone 2: Custom Object Creation

Activity 1: Create Event Object

1. Objective To create the custom database table 'Event' within the Salesforce CRM architecture to structurally store, manage, and process all event-related records for the EventForce Management System.

2. Theoretical Background & Context In the Salesforce multi-tenant cloud architecture, data is organized into database tables known as objects. While Salesforce provides robust standard objects (such as Accounts, Contacts, and standard Campaigns) out of the box, they are often insufficient for modeling

complex, specialized business domains like end-to-end event planning. Custom objects act as dedicated database tables tailored precisely to store data unique to an organization's specific application workflow.

By creating the Event__c custom object, we establish the core structural backbone of the entire EventForce system. Every subsequent component—ranging from client lookups and venue relationship modeling to automated 3-day reminder flows, approval processes for cancellations, and programmatic double-booking prevention triggers—relies directly upon this database foundation. Configuring optional features during object creation, such as reporting capabilities, activity tracking, field history auditing, and global search access, ensures that the object is fully optimized for enterprise-grade analytics, administrative oversight, and end-user efficiency right from its inception.

3. Step-by-Step Execution Guide

1. Access your Salesforce Developer Edition organization and navigate to the global Setup landing page by clicking the gear icon located in the top-right corner of the window and selecting 'Setup'.
2. Once the Setup interface loads in the browser, locate and click on the **Object Manager** tab positioned immediately next to the Home tab in the secondary navigation header.
3. In the upper-right corner of the Object Manager dashboard, click on the **Create** dropdown menu and select the **Custom Object** option to launch the New Custom Object definition form.
4. On the Custom Object Definition page, locate the **Label** field and enter the singular term 'Event' to define how the entity will appear to standard users in the user interface.
5. Move to the **Plural Label** field and enter 'Events' to ensure proper pluralization across list views, tabs, and global navigation menus.
6. In the **Record Name** section, define the primary label for the object's name field by entering 'Event Name'.
7. Select '**Text**' as the Data Type for the Record Name field to allow descriptive naming conventions for individual event records.
8. Scroll down to the **Optional Features** section and check the checkboxes for '**Allow Reports**' (enables the object data to be queried in custom reporting engines), '**Allow Activities**' (allows tasks and calendar events to be logged against records), and '**Track Field History**' (maintains an audit trail of field modifications).

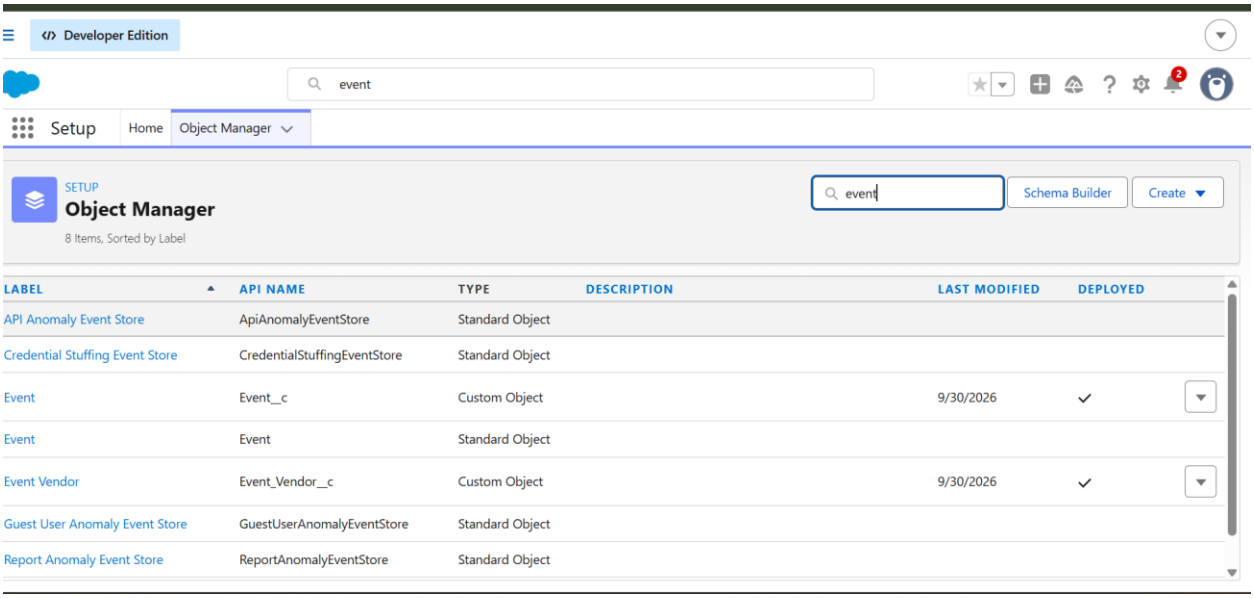
9. In the **Deployment Status** section, verify and ensure that **'Deployed'** is actively selected so that the custom object is immediately accessible to authorized users and profiles.
10. Move to the **Search Status** section and check the box for **'Allow Search'** to include this object and its records in the global Salesforce search indexing mechanism.
11. In the **Object Creation Options** section, check the box for **'Add Notes and Attachments related list to default page layout'** to provision document management capabilities for event contracts and files.
12. Review all configured parameters for accuracy and click the **'Save'** button at the top or bottom of the form to commit the custom object definition to the database metadata.

4. Verification & Testing To verify that the custom object was successfully created and provisioned, navigate back to the main **Object Manager** tab in Salesforce Setup. Use the Quick Find search box on the left-hand sidebar to search for the term 'Event'. The Event object should appear prominently in the object list. Clicking on it will reveal its technical metadata configuration, confirming its system-assigned API Name as Event__c (designating it as a custom object entity) along with the operational parameters established during creation.

5. Best Practices & Troubleshooting

- **Always Enable Reporting Early:** Always check the 'Allow Reports' feature option during initial object creation. While standard fields can be modified later, forgetting to enable reporting during object creation can cause friction when constructing analytics dashboards and tracking operational metrics down the line.
- **Global Search Optimization:** Ensure 'Allow Search' is checked on all custom objects. This prevents user frustration by enabling operational managers and coordinators to quickly locate specific records from any page in the CRM using the universal top search bar.
- **Naming Convention Standards:** Stick strictly to singular naming conventions for the object Label (e.g., 'Event') and plural naming conventions for the Plural Label (e.g., 'Events'). Salesforce utilizes these values dynamically to construct UI tabs, list views, and related list headers; mismatched plurals degrade the professional presentation of the platform.
- **Troubleshooting API Naming Collisions:** If Salesforce appends an unexpected suffix (such as Event__1c), it indicates that an object with the API name Event already exists in the namespace schema (such as standard system

objects). Ensure your custom labels uniquely identify your application context.



Activity 2: Create Client Object

1. Objective To create the custom database table 'Client' within the Salesforce CRM architecture to structurally store, manage, and process all customer and stakeholder contact records for the EventForce Management System.

2. Theoretical Background & Context In the Salesforce cloud platform, enterprise data modeling frequently requires isolated entities to segregate distinct functional domains. While standard objects like standard Accounts or Contacts exist within the CRM ecosystem, they often carry restrictive native behaviors, predefined page layouts, or mandatory fields that misalign with specialized industry workflows. Creating a dedicated custom object for clients allows developers to define a clean, uncompromised data schema tailored strictly to the business requirements of event planning.

By provisioning the Client__c custom object, we establish the relational stakeholder anchor for the entire application. This entity acts as the parent record for upcoming event bookings, financial budgeting traces, and post-event feedback metrics. Ensuring that optional features—such as activity logging, reporting engines, field

history tracking, and notes management—are enabled right at creation provides coordinators with a comprehensive, 360-degree view of every customer interaction from a single unified interface.

3. Step-by-Step Execution Guide

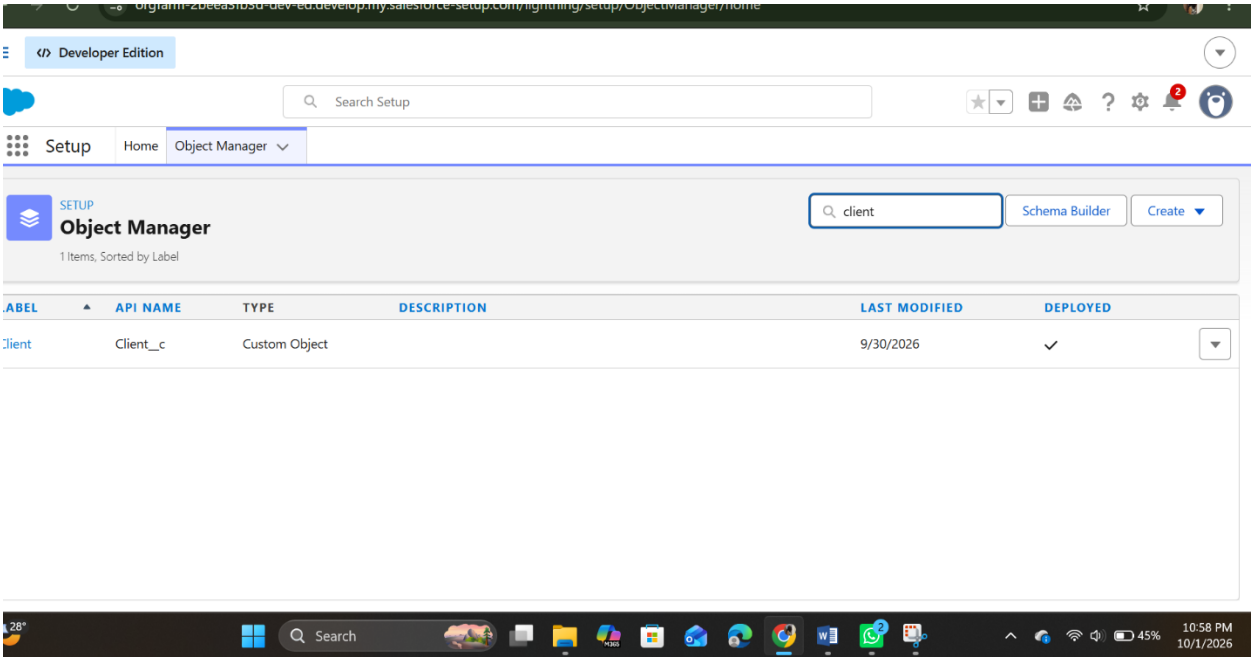
1. Access your Salesforce Developer Edition environment and navigate to the global Setup landing page by clicking the gear icon located in the top-right corner of the window and selecting 'Setup'.
2. Once the Setup interface loads in the browser, locate and click on the **Object Manager** tab positioned immediately next to the Home tab in the secondary navigation header.
3. In the upper-right corner of the Object Manager dashboard, click on the **Create** dropdown menu and select the **Custom Object** option to launch the New Custom Object definition form.
4. On the Custom Object Definition page, locate the **Label** field and enter the singular term 'Client' to define how the customer entity will appear to operational users.
5. Move to the **Plural Label** field and enter 'Clients' to ensure proper grammatical pluralization across tabs, global search layouts, and report builders.
6. In the **Record Name** section, define the primary label for the object's identity field by entering 'Client Name'.
7. Select '**Text**' as the Data Type for the Record Name field to allow full customer names to be entered as primary record identifiers.
8. Scroll down to the **Optional Features** section and check the checkboxes for '**Allow Reports**' (enables client data integration into reporting and analytics tools), '**Allow Activities**' (allows tasks, emails, and calls to be logged against client accounts), and '**Track Field History**' (maintains a detailed audit trail of profile modifications).
9. In the **Deployment Status** section, verify and ensure that '**Deployed**' is actively selected so that the custom object and its associated tabs are immediately accessible to active system users.
10. Move to the **Search Status** section and check the box for '**Allow Search**' to include this object and its individual client records in the global Salesforce search index.
11. In the **Object Creation Options** section, check the box for '**Add Notes and Attachments related list to default page layout**' to provision document management capabilities for client agreements and communication files.

12. Review all configured parameters for accuracy and click the **'Save'** button at the top or bottom of the form to commit the custom object definition to the metadata database.

4. Verification & Testing To verify that the custom object was successfully created and provisioned, navigate back to the main **Object Manager** tab in Salesforce Setup. Use the Quick Find search box on the left-hand sidebar to search for the term 'Client'. The Client object should appear prominently in the object list. Clicking on it will display its technical metadata configuration, confirming its system-assigned API Name as Client__c (designating it as a custom database entity) alongside the operational parameters established during creation.

5. Best Practices & Troubleshooting

- **Always Enable Reporting Early:** Always check the 'Allow Reports' feature option during initial object creation. While standard fields can be modified later, forgetting to enable reporting during object creation can block dashboard creation and pipeline analysis down the line.
- **Global Search Optimization:** Ensure 'Allow Search' is checked on all custom objects. This prevents user frustration by enabling operational managers and coordinators to quickly locate specific records from any page in the CRM using the universal top search bar.
- **Naming Convention Standards:** Stick strictly to singular naming conventions for the object Label (e.g., 'Client') and plural naming conventions for the Plural Label (e.g., 'Clients'). Salesforce utilizes these values dynamically to construct UI tabs, list views, and related list headers; mismatched plurals degrade the professional presentation of the platform.
- **Troubleshooting API Naming Collisions:** If Salesforce appends an unexpected suffix (such as Client__1c), it indicates that an object with the API name Client already exists in the namespace schema. Ensure your custom labels uniquely identify your application context.



Activity 3: Create Vendor Object

1. Objective To create the custom database table 'Vendor' within the Salesforce CRM architecture to structurally store, manage, and process all third-party service provider and contractor records for the EventForce Management System.

2. Theoretical Background & Context In enterprise-grade CRM implementations, managing external supply chains and contractor ecosystems requires structured, isolated entities. While standard objects like standard Accounts can represent companies, event planning operations demand specialized tracking of vendor-specific attributes, service offerings, and operational availability status. Custom objects provide the ideal database schema to isolate third-party contractor data from internal customer records.

By provisioning the Vendor__c custom object, we establish the supplier-side structural anchor for the application. This entity acts as the foundational building block that will later link to multiple events via a Many-to-Many junction object architecture. Ensuring that optional features—such as activity tracking for contract discussions, reporting tools for vendor performance analysis, and history auditing—

are enabled right at object creation ensures seamless integration into the broader EventForce ecosystem.

3. Step-by-Step Execution Guide

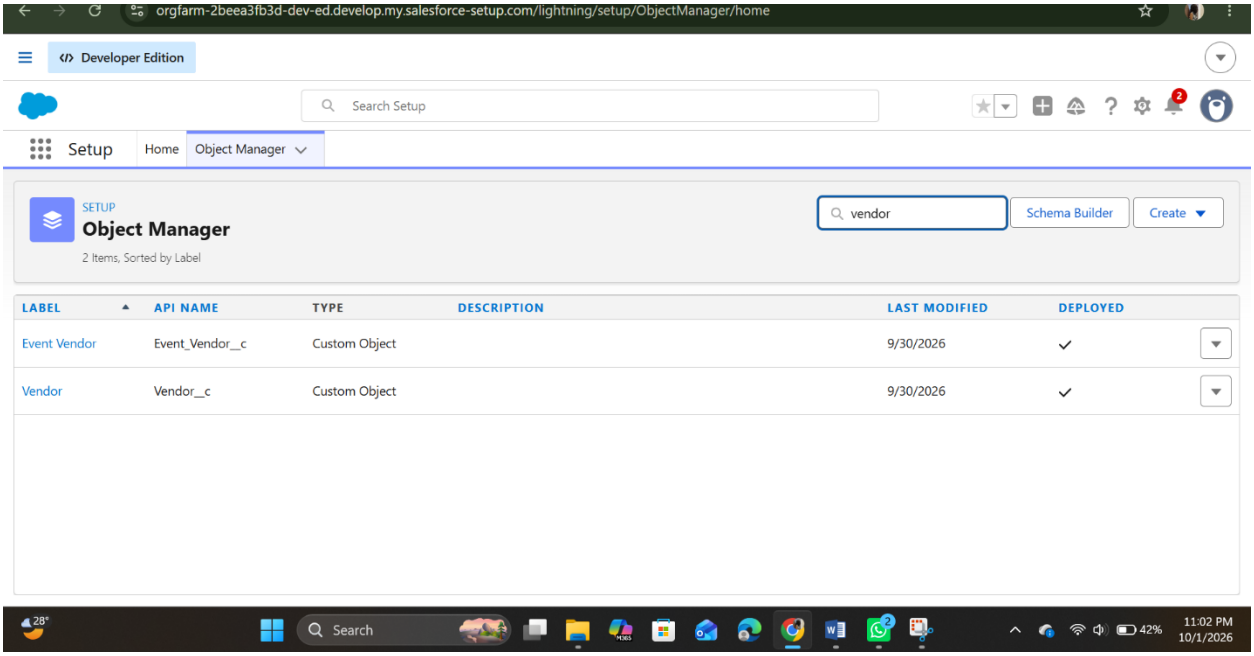
1. Access your Salesforce Developer Edition environment and navigate to the global Setup landing page by clicking the gear icon located in the top-right corner of the window and selecting 'Setup'.
2. Once the Setup interface loads in the browser, locate and click on the **Object Manager** tab positioned immediately next to the Home tab in the secondary navigation header.
3. In the upper-right corner of the Object Manager dashboard, click on the **Create** dropdown menu and select the **Custom Object** option to launch the New Custom Object definition form.
4. On the Custom Object Definition page, locate the **Label** field and enter the singular term 'Vendor' to define how the contractor entity will appear to operational users.
5. Move to the **Plural Label** field and enter 'Vendors' to ensure proper grammatical pluralization across tabs, global search layouts, and report builders.
6. In the **Record Name** section, define the primary label for the object's identity field by entering 'Vendor Name'.
7. Select '**Text**' as the Data Type for the Record Name field to allow enterprise vendor company names or individual contractor names to be entered as primary record identifiers.
8. Scroll down to the **Optional Features** section and check the checkboxes for '**Allow Reports**' (enables vendor data integration into reporting and supply chain analytics tools), '**Allow Activities**' (allows tasks, emails, and logistical calls to be logged against vendor accounts), and '**Track Field History**' (maintains a detailed audit trail of profile modifications).
9. In the **Deployment Status** section, verify and ensure that '**Deployed**' is actively selected so that the custom object and its associated tabs are immediately accessible to active system users.
10. Move to the **Search Status** section and check the box for '**Allow Search**' to include this object and its individual vendor records in the global Salesforce search index.
11. In the **Object Creation Options** section, check the box for '**Add Notes and Attachments related list to default page layout**' to provision document management capabilities for vendor contracts, service level agreements (SLAs), and catalogs.

12. Review all configured parameters for accuracy and click the **'Save'** button at the top or bottom of the form to commit the custom object definition to the metadata database.

4. Verification & Testing To verify that the custom object was successfully created and provisioned, navigate back to the main **Object Manager** tab in Salesforce Setup. Use the Quick Find search box on the left-hand sidebar to search for the term 'Vendor'. The Vendor object should appear prominently in the object list. Clicking on it will display its technical metadata configuration, confirming its system-assigned API Name as Vendor__c (designating it as a custom database entity) alongside the operational parameters established during creation.

5. Best Practices & Troubleshooting

- **Always Enable Reporting Early:** Always check the 'Allow Reports' feature option during initial object creation. While standard fields can be modified later, forgetting to enable reporting during object creation can block dashboard creation and vendor performance tracking down the line.
- **Global Search Optimization:** Ensure 'Allow Search' is checked on all custom objects. This prevents user frustration by enabling operational managers and coordinators to quickly locate specific contractor records from any page in the CRM using the universal top search bar.
- **Naming Convention Standards:** Stick strictly to singular naming conventions for the object Label (e.g., 'Vendor') and plural naming conventions for the Plural Label (e.g., 'Vendors'). Salesforce utilizes these values dynamically to construct UI tabs, list views, and related list headers; mismatched plurals degrade the professional presentation of the platform.
- **Troubleshooting API Naming Collisions:** If Salesforce appends an unexpected suffix (such as Vendor__1c), it indicates that an object with the API name Vendor already exists in the namespace schema. Ensure your custom labels uniquely identify your application context.



Activity 4: Create Venue Object

1. Objective To create the custom database table 'Venue' within the Salesforce CRM architecture to structurally store, manage, and process all physical location, capacity, and spatial reservation records for the EventForce Management System.

2. Theoretical Background & Context In complex CRM engineering, specialized business domains require isolated physical asset tracking entities. While standard objects exist for standard accounts or generic mapping tools, managing physical event spaces demands custom attributes such as spatial capacity limits, location URLs, address specifics, and real-time availability tracking. Custom objects provide the necessary database tables to isolate physical infrastructure data from internal client accounts and administrative user profiles.

By provisioning the Venue__c custom object, we establish the spatial structural anchor for the application. This entity acts as the foundational target for event location lookups and serves as the primary data model evaluated by programmatic double-booking prevention triggers. Ensuring that optional features—such as activity logging for site inspections, reporting tools for location utilization analysis,

and history auditing—are enabled right at object creation ensures seamless integration into the broader EventForce ecosystem.

3. Step-by-Step Execution Guide

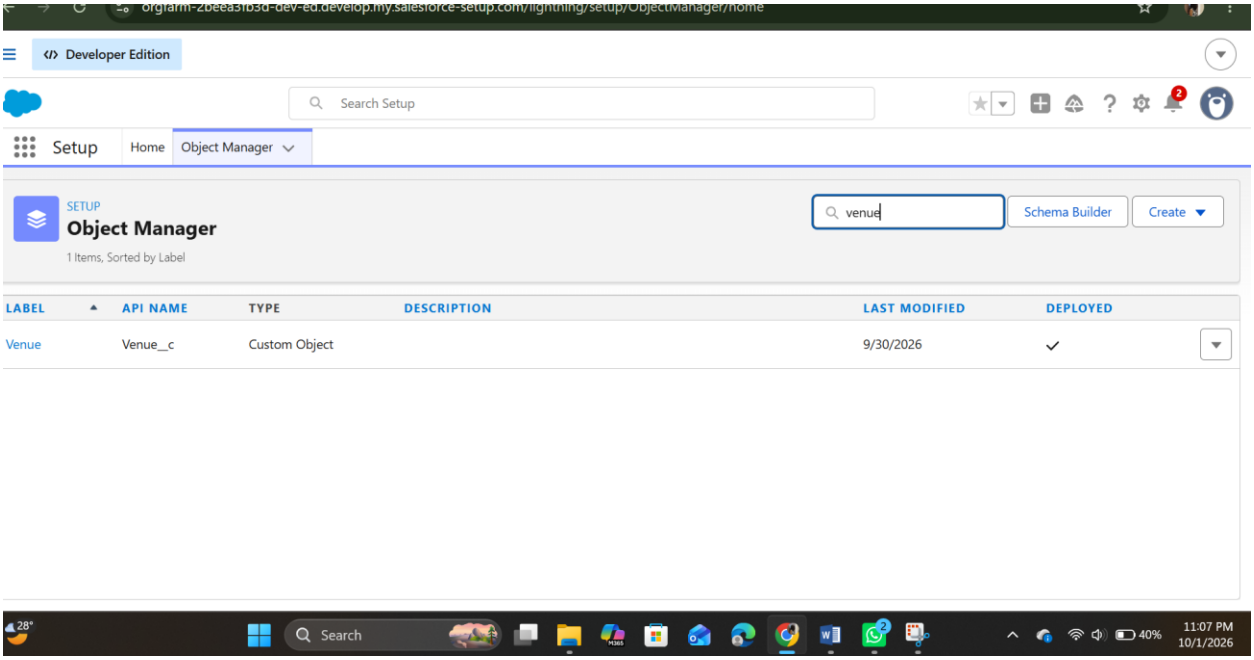
1. Access your Salesforce Developer Edition environment and navigate to the global Setup landing page by clicking the gear icon located in the top-right corner of the window and selecting 'Setup'.
2. Once the Setup interface loads in the browser, locate and click on the **Object Manager** tab positioned immediately next to the Home tab in the secondary navigation header.
3. In the upper-right corner of the Object Manager dashboard, click on the **Create** dropdown menu and select the **Custom Object** option to launch the New Custom Object definition form.
4. On the Custom Object Definition page, locate the **Label** field and enter the singular term 'Venue' to define how the location entity will appear to operational users.
5. Move to the **Plural Label** field and enter 'Venues' to ensure proper grammatical pluralization across tabs, global search layouts, and report builders.
6. In the **Record Name** section, define the primary label for the object's identity field by entering 'Venue Name'.
7. Select '**Text**' as the Data Type for the Record Name field to allow enterprise venue facility names or banquet hall designations to be entered as primary record identifiers.
8. Scroll down to the **Optional Features** section and check the checkboxes for '**Allow Reports**' (enables venue data integration into reporting and location utilization analytics tools), '**Allow Activities**' (allows tasks, site walk-throughs, and logistical calls to be logged against venue accounts), and '**Track Field History**' (maintains a detailed audit trail of profile modifications).
9. In the **Deployment Status** section, verify and ensure that '**Deployed**' is actively selected so that the custom object and its associated tabs are immediately accessible to active system users.
10. Move to the **Search Status** section and check the box for '**Allow Search**' to include this object and its individual venue records in the global Salesforce search index.
11. In the **Object Creation Options** section, check the box for '**Add Notes and Attachments related list to default page layout**' to provision document management capabilities for venue blueprints, lease contracts, and facility guidelines.

12. Review all configured parameters for accuracy and click the **'Save'** button at the top or bottom of the form to commit the custom object definition to the metadata database.

4. Verification & Testing To verify that the custom object was successfully created and provisioned, navigate back to the main **Object Manager** tab in Salesforce Setup. Use the Quick Find search box on the left-hand sidebar to search for the term 'Venue'. The Venue object should appear prominently in the object list. Clicking on it will display its technical metadata configuration, confirming its system-assigned API Name as Venue__c (designating it as a custom database entity) alongside the operational parameters established during creation.

5. Best Practices & Troubleshooting

- **Always Enable Reporting Early:** Always check the 'Allow Reports' feature option during initial object creation. While standard fields can be modified later, forgetting to enable reporting during object creation can block dashboard creation and spatial analytics tracking down the line.
- **Global Search Optimization:** Ensure 'Allow Search' is checked on all custom objects. This prevents user frustration by enabling operational managers and coordinators to quickly locate specific facility records from any page in the CRM using the universal top search bar.
- **Naming Convention Standards:** Stick strictly to singular naming conventions for the object Label (e.g., 'Venue') and plural naming conventions for the Plural Label (e.g., 'Venues'). Salesforce utilizes these values dynamically to construct UI tabs, list views, and related list headers; mismatched plurals degrade the professional presentation of the platform.
- **Troubleshooting API Naming Collisions:** If Salesforce appends an unexpected suffix (such as Venue__1c), it indicates that an object with the API name Venue already exists in the namespace schema. Ensure your custom labels uniquely identify your application context.



Activity 5: Create Feedback Object

1. Objective To create the custom database table 'Feedback' within the Salesforce CRM architecture to structurally store, manage, and process post-event customer ratings, reviews, and client satisfaction metrics for the EventForce Management System.

2. Theoretical Background & Context In enterprise CRM implementations, closing the operational feedback loop is essential for measuring service quality, tracking vendor execution success, and driving long-term customer retention. While standard system objects manage transactional sales or accounts, specialized performance monitoring requires dedicated entities to capture qualitative and quantitative client responses following project delivery. Utilizing an Auto Number data type for the primary record identifier streamlines administration by ensuring every feedback entry is automatically indexed with a uniform, sequential numbering structure.

By provisioning the Feedback__c custom object, we establish the quality assurance anchor for the application. This entity acts as a child record that links both to the Client__c who experienced the service and the specific Event__c being evaluated,

regulated later by specialized lookup filters. Ensuring that optional features—such as reporting tools for satisfaction trend analysis, activity logging for follow-up communications, and history tracking—are enabled right at object creation ensures seamless integration into the broader EventForce analytics ecosystem.

3. Step-by-Step Execution Guide

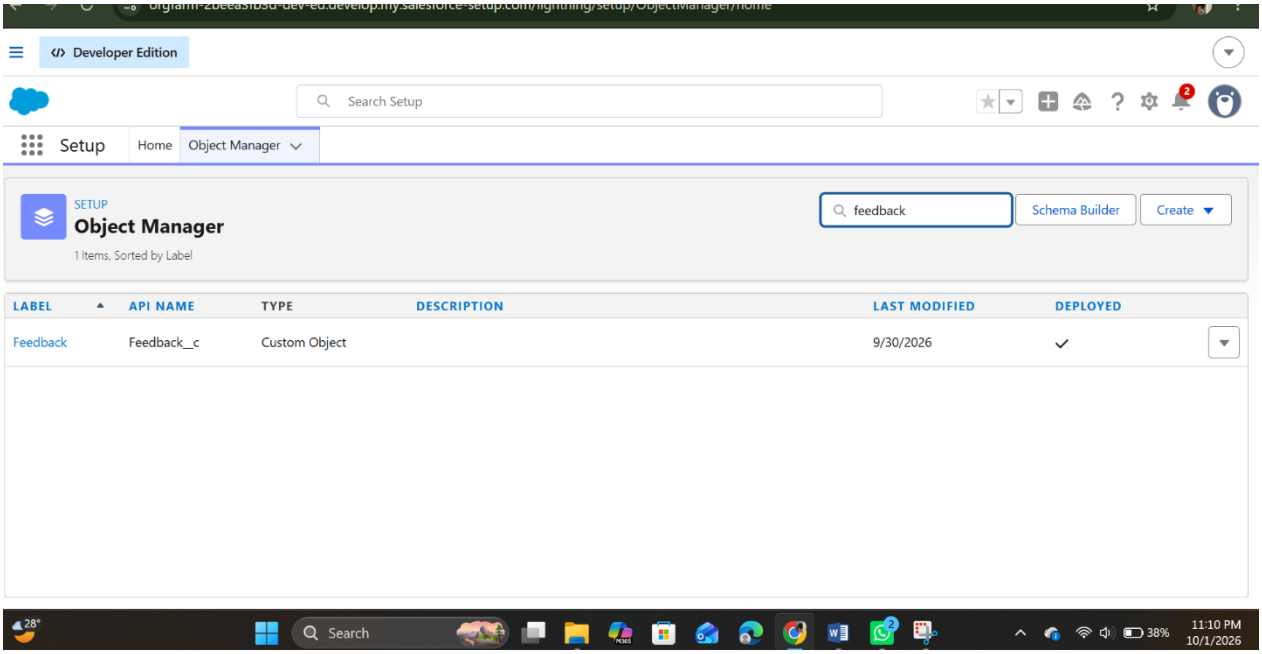
1. Access your Salesforce Developer Edition environment and navigate to the global Setup landing page by clicking the gear icon located in the top-right corner of the window and selecting 'Setup'.
2. Once the Setup interface loads in the browser, locate and click on the **Object Manager** tab positioned immediately next to the Home tab in the secondary navigation header.
3. In the upper-right corner of the Object Manager dashboard, click on the **Create** dropdown menu and select the **Custom Object** option to launch the New Custom Object definition form.
4. On the Custom Object Definition page, locate the **Label** field and enter the singular term 'Feedback' to define how the customer satisfaction entity will appear to operational users.
5. Move to the **Plural Label** field and enter 'Feedbacks' to ensure proper grammatical pluralization across tabs, global search layouts, and report builders.
6. In the **Record Name** section, define the primary label for the object's identity field by entering 'Feedback ID'.
7. Select **'Auto Number'** as the Data Type for the Record Name field to enable automatic sequential numbering of incoming review records.
8. Under the auto number configuration settings, enter the strict Display Format as 'F-{0000}' and set the Starting Number to '0' to generate uniform identifiers like F-0001, F-0002, etc..
9. Scroll down to the **Optional Features** section and check the checkboxes for **'Allow Reports'** (enables feedback data integration into customer satisfaction analytics tools), **'Allow Activities'** (allows tasks and follow-up calls to be logged against feedback entries), and **'Track Field History'** (maintains a detailed audit trail of rating modifications).
10. In the **Deployment Status** section, verify and ensure that **'Deployed'** is actively selected so that the custom object and its associated tabs are immediately accessible to active system users.
11. Move to the **Search Status** section and check the box for **'Allow Search'** to include this object and its individual review records in the global Salesforce search index.

12. In the **Object Creation Options** section, check the box for **'Add Notes and Attachments related list to default page layout'** to provision document management capabilities for client testimonial letters or media attachments.
13. Review all configured parameters for accuracy and click the **'Save'** button at the top or bottom of the form to commit the custom object definition to the metadata database.

4. Verification & Testing To verify that the custom object was successfully created and provisioned, navigate back to the main **Object Manager** tab in Salesforce Setup. Use the Quick Find search box on the left-hand sidebar to search for the term 'Feedback'. The Feedback object should appear prominently in the object list. Clicking on it will display its technical metadata configuration, confirming its system-assigned API Name as Feedback__c (designating it as a custom database entity) alongside the operational parameters established during creation.

5. Best Practices & Troubleshooting

- **Always Enable Reporting Early:** Always check the 'Allow Reports' feature option during initial object creation. While standard fields can be modified later, forgetting to enable reporting during object creation can block dashboard creation and customer satisfaction analytics tracking down the line.
- **Global Search Optimization:** Ensure 'Allow Search' is checked on all custom objects. This prevents user frustration by enabling operational managers and coordinators to quickly locate specific review records from any page in the CRM using the universal top search bar.
- **Naming Convention Standards:** Stick strictly to singular naming conventions for the object Label (e.g., 'Feedback') and plural naming conventions for the Plural Label (e.g., 'Feedbacks'). Salesforce utilizes these values dynamically to construct UI tabs, list views, and related list headers; mismatched plurals degrade the professional presentation of the platform.
- **Troubleshooting API Naming Collisions:** If Salesforce appends an unexpected suffix (such as Feedback__1c), it indicates that an object with the API name Feedback already exists in the namespace schema. Ensure your custom labels uniquely identify your application context.



Milestone 3: Custom Tabs Setup

Activity 1: Create a Custom Tab for Event

1. Objective To create a user interface tab for the Event object, allowing users to access and manage Event records.

2. Theoretical Background & Context While an object defines the database table within the Salesforce schema, a Custom Tab is the essential graphical user interface component that exposes that table to system users. Without a dedicated tab provisioned for the object, operational users cannot easily view custom list views, create new event records, or access the data directly from the Salesforce App Launcher. Creating custom object tabs transforms backend database structures into interactive, accessible workspaces for event coordinators.

3. Step-by-Step Execution Guide

1. From the global Salesforce Setup landing page, click on the **Home** tab in the main navigation menu.
2. Locate the Quick Find search box on the left-hand sidebar, enter the term 'Tabs', and select the **Tabs** option under the User Interface menu.

3. Scroll down to the **Custom Object Tabs** section and click the **New** button to initiate the custom tab wizard.
4. From the primary **Object** dropdown selection list, locate and select **'Event'**.
5. Click the magnifying glass icon next to the **Tab Style** field to open the color palette and icon selector, and choose an icon that conceptually matches an event management context (e.g., a calendar or flag icon).
6. Click **'Next'** to proceed to the profile accessibility configuration stage.
7. Leave the profile visibility set to **'Default On'** for all standard and custom user profiles, ensuring system accessibility (access permissions will be restricted later via fine-grained profile and permission set controls). Click **'Next'**.
8. Ensure the tab is included across your target custom applications (such as the default inclusion settings) and click **'Save'** to finalize tab provisioning.

4. Verification & Testing To verify that the custom tab is fully operational, click on the **App Launcher** (represented by the nine-dot grid icon in the top-left corner) and search for 'Event'. You should see the Event tab listed under the available items section. Clicking on it will successfully open the recently viewed list view interface for the Event object.

5. Best Practices & Troubleshooting

- Choose intuitive, contextually relevant icons for your custom tab styles to maximize user adoption, visual appeal, and ease of navigation across the workspace.
- **Troubleshooting Missing Tabs:** If a custom tab fails to appear inside the App Launcher for specific users, verify their individual Profile settings to ensure that the object tab setting is not accidentally configured as 'Tab Hidden'.

Activity 2: Create a Custom Tab for Client

1. Objective To create a user interface tab for the Client object, allowing users to access and manage Client records.

2. Theoretical Background & Context While an object defines the database table within the Salesforce schema, a Custom Tab is the essential graphical user interface component that exposes that table to system users. Without a dedicated tab provisioned for the object, operational users cannot easily view custom list views, create new client records, or access customer data directly from the Salesforce App

Launcher. Creating custom object tabs transforms backend database structures into interactive, accessible workspaces for event coordinators.

3. Step-by-Step Execution Guide

1. From the global Salesforce Setup landing page, click on the **Home** tab in the main navigation menu.
2. Locate the Quick Find search box on the left-hand sidebar, enter the term 'Tabs', and select the **Tabs** option under the User Interface menu.
3. Scroll down to the **Custom Object Tabs** section and click the **New** button to initiate the custom tab wizard.
4. From the primary **Object** dropdown selection list, locate and select **'Client'**.
5. Click the magnifying glass icon next to the **Tab Style** field to open the color palette and icon selector, and choose an icon that conceptually matches a customer or user profile context.
6. Click **'Next'** to proceed to the profile accessibility configuration stage.
7. Leave the profile visibility set to **'Default On'** for all standard and custom user profiles, ensuring system accessibility. Click **'Next'**.
8. Ensure the tab is included across your target custom applications and click **'Save'** to finalize tab provisioning.

4. Verification & Testing To verify that the custom tab is fully operational, click on the **App Launcher** (represented by the nine-dot grid icon in the top-left corner) and search for 'Client'. You should see the Client tab listed under the available items section. Clicking on it will successfully open the recently viewed list view interface for the Client object.

5. Best Practices & Troubleshooting

- Choose intuitive, contextually relevant icons for your custom tab styles to maximize user adoption, visual appeal, and ease of navigation across the workspace.
- **Troubleshooting Missing Tabs:** If a custom tab fails to appear inside the App Launcher for specific users, verify their individual Profile settings to ensure that the object tab setting is not accidentally configured as 'Tab Hidden'.

Activity 3: Create a Custom Tab for Vendor

1. Objective To create a user interface tab for the Vendor object, allowing users to access and manage Vendor records.

2. Theoretical Background & Context While an object defines the database table within the Salesforce schema, a Custom Tab is the essential graphical user interface component that exposes that table to system users. Without a dedicated tab provisioned for the object, operational users cannot easily view custom list views, create new vendor records, or access contractor data directly from the Salesforce App Launcher. Creating custom object tabs transforms backend database structures into interactive, accessible workspaces for event coordinators.

3. Step-by-Step Execution Guide

1. From the global Salesforce Setup landing page, click on the **Home** tab in the main navigation menu.
2. Locate the Quick Find search box on the left-hand sidebar, enter the term 'Tabs', and select the **Tabs** option under the User Interface menu.
3. Scroll down to the **Custom Object Tabs** section and click the **New** button to initiate the custom tab wizard.
4. From the primary **Object** dropdown selection list, locate and select '**Vendor**'.
5. Click the magnifying glass icon next to the **Tab Style** field to open the color palette and icon selector, and choose an icon that conceptually matches a service provider or truck/toolbox context.
6. Click '**Next**' to proceed to the profile accessibility configuration stage.
7. Leave the profile visibility set to '**Default On**' for all standard and custom user profiles, ensuring system accessibility. Click '**Next**'.
8. Ensure the tab is included across your target custom applications and click '**Save**' to finalize tab provisioning.

4. Verification & Testing To verify that the custom tab is fully operational, click on the **App Launcher** (represented by the nine-dot grid icon in the top-left corner) and search for 'Vendor'. You should see the Vendor tab listed under the available items section. Clicking on it will successfully open the recently viewed list view interface for the Vendor object.

5. Best Practices & Troubleshooting

- Choose intuitive, contextually relevant icons for your custom tab styles to maximize user adoption, visual appeal, and ease of navigation across the workspace.
- **Troubleshooting Missing Tabs:** If a custom tab fails to appear inside the App Launcher for specific users, verify their individual Profile settings to ensure that the object tab setting is not accidentally configured as 'Tab Hidden'.

Activity 4: Create a Custom Tab for Venue

1. Objective To create a user interface tab for the Venue object, allowing users to access and manage Venue records.

2. Theoretical Background & Context While an object defines the database table within the Salesforce schema, a Custom Tab is the essential graphical user interface component that exposes that table to system users. Without a dedicated tab provisioned for the object, operational users cannot easily view custom list views, create new venue records, or access spatial facility data directly from the Salesforce App Launcher. Creating custom object tabs transforms backend database structures into interactive, accessible workspaces for event coordinators.

3. Step-by-Step Execution Guide

1. From the global Salesforce Setup landing page, click on the **Home** tab in the main navigation menu.
2. Locate the Quick Find search box on the left-hand sidebar, enter the term 'Tabs', and select the **Tabs** option under the User Interface menu.
3. Scroll down to the **Custom Object Tabs** section and click the **New** button to initiate the custom tab wizard.
4. From the primary **Object** dropdown selection list, locate and select **'Venue'**.
5. Click the magnifying glass icon next to the **Tab Style** field to open the color palette and icon selector, and choose an icon that conceptually matches a building, map pin, or location context.
6. Click **'Next'** to proceed to the profile accessibility configuration stage.
7. Leave the profile visibility set to **'Default On'** for all standard and custom user profiles, ensuring system accessibility. Click **'Next'**.
8. Ensure the tab is included across your target custom applications and click **'Save'** to finalize tab provisioning.

4. Verification & Testing To verify that the custom tab is fully operational, click on the **App Launcher** (represented by the nine-dot grid icon in the top-left corner) and search for 'Venue'. You should see the Venue tab listed under the available items section. Clicking on it will successfully open the recently viewed list view interface for the Venue object.

5. Best Practices & Troubleshooting

- Choose intuitive, contextually relevant icons for your custom tab styles to maximize user adoption, visual appeal, and ease of navigation across the workspace.
- **Troubleshooting Missing Tabs:** If a custom tab fails to appear inside the App Launcher for specific users, verify their individual Profile settings to ensure that the object tab setting is not accidentally configured as 'Tab Hidden'.

Activity 5: Create a Custom Tab for Feedback

1. Objective To create a user interface tab for the Feedback object, allowing users to access and manage Feedback records.

2. Theoretical Background & Context While an object defines the database table within the Salesforce schema, a Custom Tab is the essential graphical user interface component that exposes that table to system users. Without a dedicated tab provisioned for the object, operational users cannot easily view custom list views, create new feedback review records, or access customer satisfaction analytics directly from the Salesforce App Launcher. Creating custom object tabs transforms backend database structures into interactive, accessible workspaces for event coordinators.

3. Step-by-Step Execution Guide

1. From the global Salesforce Setup landing page, click on the **Home** tab in the main navigation menu.
2. Locate the Quick Find search box on the left-hand sidebar, enter the term 'Tabs', and select the **Tabs** option under the User Interface menu.
3. Scroll down to the **Custom Object Tabs** section and click the **New** button to initiate the custom tab wizard.
4. From the primary **Object** dropdown selection list, locate and select '**Feedback**'.

5. Click the magnifying glass icon next to the **Tab Style** field to open the color palette and icon selector, and choose an icon that conceptually matches a star rating, clipboard, or review context.
6. Click **'Next'** to proceed to the profile accessibility configuration stage.
7. Leave the profile visibility set to **'Default On'** for all standard and custom user profiles, ensuring system accessibility. Click **'Next'**.
8. Ensure the tab is included across your target custom applications and click **'Save'** to finalize tab provisioning.

4. Verification & Testing To verify that the custom tab is fully operational, click on the **App Launcher** (represented by the nine-dot grid icon in the top-left corner) and search for 'Feedback'. You should see the Feedback tab listed under the available items section. Clicking on it will successfully open the recently viewed list view interface for the Feedback object.

5. Best Practices & Troubleshooting

- Choose intuitive, contextually relevant icons for your custom tab styles to maximize user adoption, visual appeal, and ease of navigation across the workspace.
- **Troubleshooting Missing Tabs:** If a custom tab fails to appear inside the App Launcher for specific users, verify their individual Profile settings to ensure that the object tab setting is not accidentally configured as 'Tab Hidden'.

Milestone 4: Fields & Relationships

Activity 1: Create Fields for Event

1. Objective To define the attributes and data points tracked on the Event object.

2. Theoretical Background & Context Fields represent the columns in our database table. Creating appropriate data types (like Picklists for standardized reporting, or Formulas for automated calculations) ensures data integrity. For Event, we are implementing: Event Type (Picklist), Event Date (Date), Event Status (Picklist), Event Budget (Formula).

3. Step-by-Step Execution Guide

1. From Setup, go to Object Manager and select 'Event'.

2. Click on 'Fields & Relationships' in the left sidebar.
3. Click the 'New' button.
4. Select the appropriate Data Type (e.g., Picklist, Date, Formula) based on the requirement and click 'Next'.
5. Enter the Field Label and configure specific attributes (like Picklist values entered on separate lines).
6. Establish field-level security by leaving default profile access checked. Click 'Next'.
7. Add the field to the default Page Layout. Click 'Save & New' to continue adding fields, or 'Save' on the final field.

4. Verification & Testing Open an existing Event record (or click 'New' from the Event tab) and verify that all newly created fields appear on the page layout with the correct data types.

5. Best Practices & Troubleshooting

- Use Picklists instead of Text fields wherever possible to ensure clean data for reporting.
- When creating Formula fields (like Event Budget), always use the 'Check Syntax' button before saving to prevent deployment errors.
- Provide Help Text for complex fields to guide end-users during data entry.

Activity 2: Create Fields for Client

1. Objective To define the attributes and data points tracked on the Client object.

2. Theoretical Background & Context Fields represent the columns in our database table. Creating appropriate data types (like Picklists for standardized reporting, or Formulas for automated calculations) ensures data integrity. For Client, we are implementing: Email (Email), Phone (Phone), Address (Text Area), Country (Picklist), City (Picklist).

3. Step-by-Step Execution Guide

1. From Setup, go to Object Manager and select 'Client'.
2. Click on 'Fields & Relationships' in the left sidebar.
3. Click the 'New' button.

4. Select the appropriate Data Type (e.g., Picklist, Date, Formula) based on the requirement and click 'Next'.
5. Enter the Field Label and configure specific attributes (like Picklist values entered on separate lines).
6. Establish field-level security by leaving default profile access checked. Click 'Next'.
7. Add the field to the default Page Layout. Click 'Save & New' to continue adding fields, or 'Save' on the final field.

4. Verification & Testing Open an existing Client record (or click 'New' from the Client tab) and verify that all newly created fields appear on the page layout with the correct data types.

5. Best Practices & Troubleshooting

- Use Picklists instead of Text fields wherever possible to ensure clean data for reporting.
- When creating Formula fields (like Event Budget), always use the 'Check Syntax' button before saving to prevent deployment errors.
- Provide Help Text for complex fields to guide end-users during data entry.

Activity 3: Create Fields for Vendor

1. Objective To define the attributes and data points tracked on the Vendor object.

2. Theoretical Background & Context Fields represent the columns in our database table. Creating appropriate data types (like Picklists for standardized reporting, or Formulas for automated calculations) ensures data integrity. For Vendor, we are implementing: Service Type (Multi-Select Picklist), Email (Email), Phone (Phone), Status (Picklist).

3. Step-by-Step Execution Guide

1. From Setup, go to Object Manager and select 'Vendor'.
2. Click on 'Fields & Relationships' in the left sidebar.
3. Click the 'New' button.
4. Select the appropriate Data Type (e.g., Picklist, Date, Formula) based on the requirement and click 'Next'.

5. Enter the Field Label and configure specific attributes (like Picklist values entered on separate lines).
6. Establish field-level security by leaving default profile access checked. Click 'Next'.
7. Add the field to the default Page Layout. Click 'Save & New' to continue adding fields, or 'Save' on the final field.

4. Verification & Testing Open an existing Vendor record (or click 'New' from the Vendor tab) and verify that all newly created fields appear on the page layout with the correct data types.

5. Best Practices & Troubleshooting

- Use Picklists instead of Text fields wherever possible to ensure clean data for reporting.
- When creating Formula fields (like Event Budget), always use the 'Check Syntax' button before saving to prevent deployment errors.
- Provide Help Text for complex fields to guide end-users during data entry.

Activity 4: Create Fields for Venue

1. Objective To define the attributes and data points tracked on the Venue object.

2. Theoretical Background & Context Fields represent the columns in our database table. Creating appropriate data types (like Picklists for standardized reporting, or Formulas for automated calculations) ensures data integrity. For Venue, we are implementing: Address (Text Area), Location (URL), Capacity (Number), Availability Status (Picklist).

3. Step-by-Step Execution Guide

1. From Setup, go to Object Manager and select 'Venue'.
2. Click on 'Fields & Relationships' in the left sidebar.
3. Click the 'New' button.
4. Select the appropriate Data Type (e.g., Picklist, Date, Formula) based on the requirement and click 'Next'.
5. Enter the Field Label and configure specific attributes (like Picklist values entered on separate lines).

6. Establish field-level security by leaving default profile access checked. Click 'Next'.
7. Add the field to the default Page Layout. Click 'Save & New' to continue adding fields, or 'Save' on the final field.

4. Verification & Testing Open an existing Venue record (or click 'New' from the Venue tab) and verify that all newly created fields appear on the page layout with the correct data types.

5. Best Practices & Troubleshooting

- Use Picklists instead of Text fields wherever possible to ensure clean data for reporting.
- When creating Formula fields (like Event Budget), always use the 'Check Syntax' button before saving to prevent deployment errors.
- Provide Help Text for complex fields to guide end-users during data entry.

Activity 5: Create Fields for Feedback

1. Objective To define the attributes and data points tracked on the Feedback object.

2. Theoretical Background & Context Fields represent the columns in our database table. Creating appropriate data types (like Picklists for standardized reporting, or Formulas for automated calculations) ensures data integrity. For Feedback, we are implementing: Rating (Picklist), Comments (Long Text).

3. Step-by-Step Execution Guide

1. From Setup, go to Object Manager and select 'Feedback'.
2. Click on 'Fields & Relationships' in the left sidebar.
3. Click the 'New' button.
4. Select the appropriate Data Type (e.g., Picklist, Date, Formula) based on the requirement and click 'Next'.
5. Enter the Field Label and configure specific attributes (like Picklist values entered on separate lines).
6. Establish field-level security by leaving default profile access checked. Click 'Next'.

7. Add the field to the default Page Layout. Click 'Save & New' to continue adding fields, or 'Save' on the final field.

4. Verification & Testing Open an existing Feedback record (or click 'New' from the Feedback tab) and verify that all newly created fields appear on the page layout with the correct data types.

5. Best Practices & Troubleshooting

- Use Picklists instead of Text fields wherever possible to ensure clean data for reporting.
- When creating Formula fields (like Event Budget), always use the 'Check Syntax' button before saving to prevent deployment errors.
- Provide Help Text for complex fields to guide end-users during data entry.

Activity 6: Create Lookups and Junction Object (EventVendor)

1. Objective To establish relational data structures between Events, Venues, Clients, and Vendors.

2. Theoretical Background & Context Salesforce uses relational fields to connect objects. A Lookup relationship creates a loose coupling (e.g., Event to Client), while a Master-Detail creates a strong parent-child bond. Because an Event can have multiple Vendors, and a Vendor can service multiple Events, we must construct a Many-to-Many relationship using a junction object (EventVendor__c).

3. Step-by-Step Execution Guide

1. Create Lookup Event -> Venue: Go to Event Object -> Fields & Relationships -> New -> Lookup Relationship. Target: Venue. Save.
2. Create Lookup Event -> Client: Repeat process targeting Client. Save.
3. Create Junction Object: Go to Object Manager -> New Custom Object. Label: Event Vendor (EventVendor__c). Save.
4. Create Master-Detail 1: On EventVendor__c -> Fields & Relationships -> New -> Master-Detail. Target: Event. Save.
5. Create Master-Detail 2: On EventVendor__c -> Fields & Relationships -> New -> Master-Detail. Target: Vendor. Save.

4. Verification & Testing Go to an Event record. You should see the 'Event Vendors' related list. Clicking 'New' on this list should prompt you to select a Vendor, proving the many-to-many link is functional.

5. Best Practices & Troubleshooting

- Always create Master-Detail fields before loading data into the junction object, as creating them on populated objects requires existing records to have a parent assigned.
- Ensure the related list labels on the parent page layouts are named intuitively (e.g., 'Assigned Vendors').

Milestone 5: Validation Rules

Activity 1: Create Validation Rule on Client

1. Objective To enforce strict database hygiene and data quality by programmatically ensuring that operational users enter a properly formatted email address when creating or updating a Client profile.

2. Theoretical Background & Context In enterprise CRM systems, data integrity is paramount. "Dirty data"—such as malformed email addresses or incomplete phone numbers—can catastrophically break downstream automation. Because the EventForce Management System relies heavily on automated, time-based email reminders (such as the 3-day pre-event client notification), an invalid email address on the Client record would cause the background Flow to fail and bounce the email.

To prevent this, Salesforce utilizes Validation Rules. A Validation Rule contains a boolean formula that is evaluated during the transactional save process, before the record is committed to the database. If the formula evaluates to **True** (meaning an error condition exists), the system aborts the save operation and immediately alerts the user. For complex string evaluations—like validating the specific alphanumeric structure of an email address—Salesforce leverages the REGEX() (Regular Expression) function, which matches the user's input against an industry-standard pattern.

3. Step-by-Step Execution Guide

1. From the global Salesforce Setup environment, navigate to the **Object Manager** tab and select the **'Client'** (Client__c) custom object.
2. On the left-hand configuration sidebar, scroll down and click on **'Validation Rules'**, then click the **'New'** button in the top-right corner to open the rule builder.
3. In the Rule Name field, enter the precise developer name 'Email_Valid_Address'. Ensure that the **'Active'** checkbox is ticked so the rule is enforced immediately upon saving.
4. Scroll down to the **Error Condition Formula** canvas. Input the following logical formula exactly as written: NOT(REGEX(Email__c, "^[a-zA-Z0-9._]+@[a-zA-Z0-9.]+\\.[a-zA-Z]{2,}$")) *Contextual Note on Logic:* The REGEX() function checks if the data entered matches the standard email structure (letters/numbers, followed by an @ symbol, a domain, a dot, and an extension). The NOT() function encapsulates it because a Validation Rule fires when the error condition evaluates to TRUE (i.e., it is TRUE that the input does NOT match the required pattern).
5. Click the **'Check Syntax'** button located below the formula canvas to compile the code and ensure there are no missing parentheses, misspelled API names, or syntax errors.
6. Scroll down to the **Error Message** section. In the text box, enter the user-facing alert: 'Please Enter Valid Email Address'.
7. Under the **Error Location** setting, select the **'Field'** radio button, and select the **'Email'** field from the adjacent dropdown menu. This ensures the error message appears directly beneath the offending input box, rather than vaguely at the top of the page.
8. Click **'Save'** to deploy the Validation Rule to the object schema.

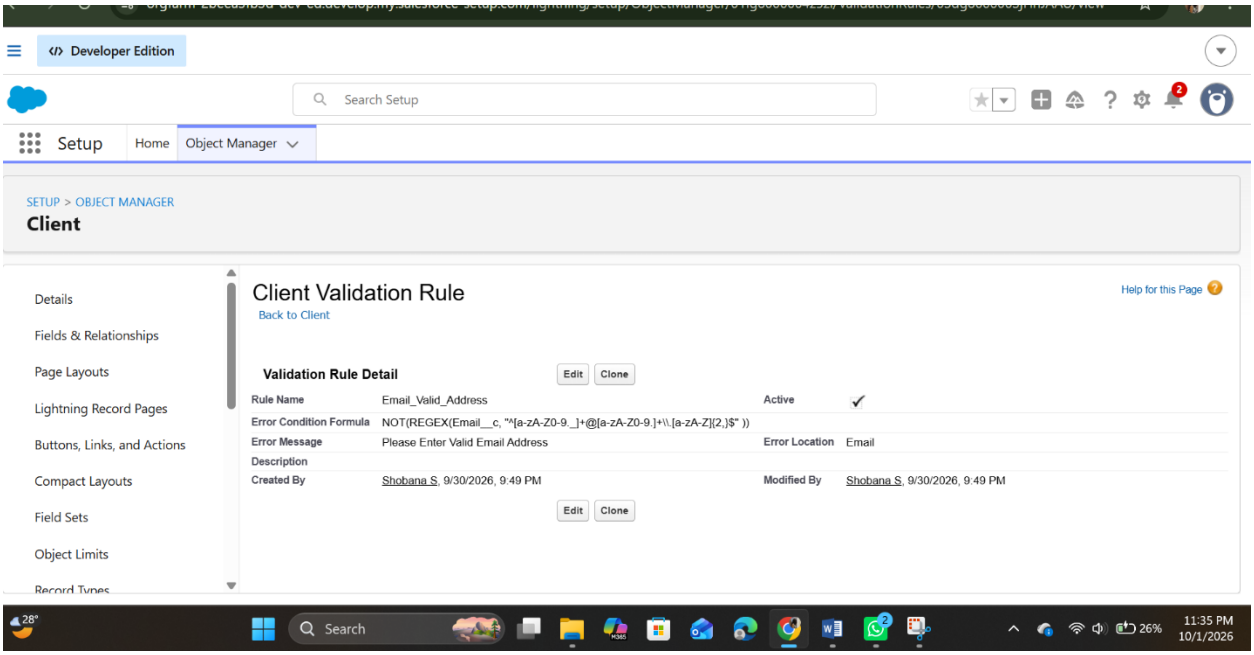
4. Verification & Testing To comprehensively test this logic, execute both a negative and a positive test:

- **Negative Testing:** Navigate to the Event Planner Lightning App, open the Clients tab, and click 'New'. Enter dummy data for the name, but intentionally enter a malformed string in the Email field (e.g., 'test@invalid' or 'john.doe.com' missing the @). Click Save. The system should successfully block the transaction, highlight the field in red, and display the error message precisely under the Email input box.
- **Positive Testing:** Correct the email address to a valid format (e.g., 'test@example.com') and click Save. The record should now save

successfully, proving the formula accurately differentiates between valid and invalid structures.

5. Best Practices & Troubleshooting

- **Leverage REGEX for Standard Formats:** Always utilize REGEX() functions for complex, standardized string validations like email addresses, international phone numbers, postal codes, or Social Security Numbers. Using basic declarative functions (like CONTAINS()) for these checks is highly prone to edge-case failures.
- **User Experience (UX) in Error Placement:** Always anchor the Error Location to the specific offending field whenever possible. Selecting 'Top of Page' can confuse users on lengthy page layouts, forcing them to hunt for the mistake.
- **Actionable Messaging:** Keep error messages polite, highly specific, and actionable. "Please Enter Valid Email Address" is far more helpful than a generic "Data Error 104".
- **Troubleshooting Syntax Errors:** If the syntax checker throws a compilation error, verify that the API name of the field (Email__c) matches your org exactly, and ensure that no trailing spaces or smart quotes (introduced by word processors) were pasted into the formula editor.



Milestone 6: Approval Process

Activity 1: Event Cancellation Approval

1. Objective To engineer and automate a strict managerial review workflow that governs the event cancellation process, ensuring that no Event record can be unilaterally marked as "Canceled" by a coordinator without formal authorization from leadership.

2. Theoretical Background & Context In enterprise event management, canceling an event is a high-risk transactional event. It carries significant financial penalties, requires the immediate release of reserved physical venues, and necessitates the dissolution of multiple third-party vendor contracts. If an Event Coordinator accidentally or maliciously changes an event status to "Canceled" without oversight, the operational fallout can be catastrophic.

To mitigate this risk, Salesforce provides the **Approval Process** automation tool. Unlike standard Record-Triggered Flows that execute blindly in the background, an Approval Process is a structured, multi-step governance engine. It enforces human-in-the-loop decision-making. When a record enters an Approval Process, Salesforce automatically "locks" the record (preventing further edits by standard users) and routes an approval request to a designated supervisor. By building the Event_Cancellation_Process, the system legally and programmatically restricts the ability to finalize a cancellation, transferring that authority exclusively to the Vendor Manager or System Administrator while automatically handling all the necessary field updates and stakeholder communications upon final decision.

3. Step-by-Step Execution Guide

Phase 1: Communication Architecture (Email Templates) Before building the logic engine, the system requires standardized communication templates.

1. Navigate to Setup, type 'Classic Email Templates' in the Quick Find box, and select it.
2. Ensure you are saving these in a **Public Folder** (e.g., "Unfiled Public Classic Email Templates") so the system can access them.
3. Click **New Template**, select **Text**, and create four distinct templates:
    - *Submission Notice:* Alerts the Manager that an event requires cancellation review. (Include merge fields for {!Event__c.Name} and {!Event__c.Event_Date__c}).

- *Client Notice:* A polite, automated notification informing the client that their event cancellation has been processed.
- *Approval Notice:* Alerts the Event Owner (Coordinator) that management approved the cancellation.
- *Rejection Notice:* Alerts the Event Owner that the cancellation was rejected and they must proceed with the event.

Phase 2: Approval Process Definition 4. In Setup, search for and select **Approval Processes**. 5. From the 'Manage Approval Processes For' dropdown, select the **Event** object. 6. Click the **Create New Approval Process** button and select **Use Standard Setup Wizard** for maximum configuration control. 7. **Name and Description:** Enter the Process Name as 'Event_Cancellation_Process'. 8. **Specify Entry Criteria:** This dictates when the process can be triggered. Set the criteria to: Event: Event Status | Equals | Pending Cancellation. (This prevents users from submitting events for approval if they are merely in the "Planned" stage).

Phase 3: Routing and Approver Designation 9. **Approver Routing:** In step 3 of the wizard, choose the option to **'Automatically assign using user'**. Click the lookup magnifying glass and explicitly select the specific Manager or Administrator user who holds cancellation authority. 10. **Approval Assignment Email Template:** Assign the *Submission Notice* email template created in Phase 1. This is the email the manager will receive when the button is clicked. 11. **Approval Page Layout:** Select the fields the manager needs to see on the approval screen to make an informed decision (e.g., Event Name, Event Budget, Client Name, Event Date).

Phase 4: Defining Automation Actions 12. **Initial Submission Actions:** By default, Salesforce locks the record. Add a new action here: **Email Alert**. Configure this alert to send the *Submission Notice* to the assigned Manager. 13. **Final Approval Actions:** This is where the actual status change occurs. * Add a **Field Update** action. Set it to update the Event Status field to 'Canceled'. * Add an **Email Alert** action. Configure it to send the *Client Notice* to the related Client's email, and the *Approval Notice* to the Event Owner. 14. **Final Rejection Actions:** * Add a **Field Update** action. Set it to revert the Event Status field back to 'Planned' or 'Rejected'. * Add an **Email Alert** action to send the *Rejection Notice* to the Event Owner. * Ensure the record is set to "Unlock" upon rejection so the coordinator can resume working on it.

Phase 5: Adding Steps and Activation 15. The wizard will prompt you to create an Approval Step. Name it 'Manager Review'. Ensure all records entering the process

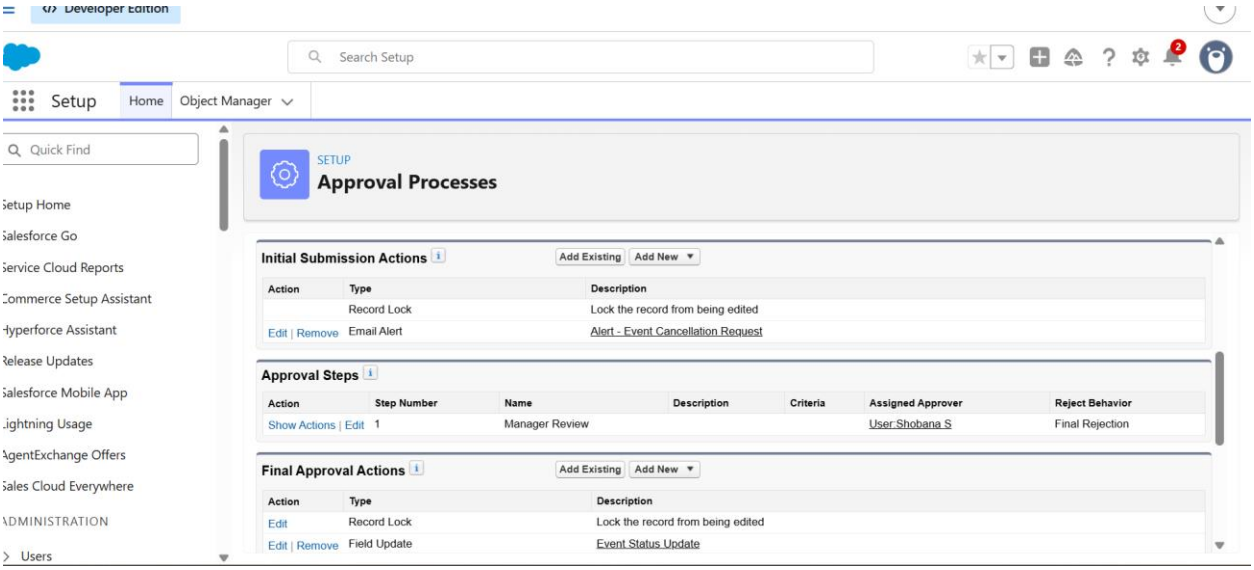
must complete this step. 16. Review the entire workflow architecture on the summary page. Once verified, click the **Activate** button.

4. Verification & Testing To guarantee the governance model functions flawlessly, simulate the end-user experience:

1. Navigate to the Event Planner Lightning App and open an existing Event record.
2. Change the Event Status to 'Pending Cancellation' and click Save.
3. Locate and click the **'Submit for Approval'** button on the top right of the page layout. (If the button is missing, edit the Event Page Layout to add it to the Salesforce Mobile and Lightning Experience Actions menu).
4. Verify that a padlock icon appears, indicating the record is locked against further edits.
5. Log in as the designated Manager (or check their email). Click the approval link, write a test comment, and click **Approve**.
6. Return to the Event record. Verify that the Status field was automatically updated to "Canceled" and that the automated emails were dispatched to both the client and the coordinator.

5. Best Practices & Troubleshooting

- **Page Layout Configuration:** The most common point of failure in user adoption is forgetting to expose the 'Submit for Approval' button. Always verify the button is present on the specific Page Layout assigned to the Event Coordinators.
- **Template Visibility:** Ensure all four email templates are saved in a Public folder and have the "Available for Use" checkbox ticked. If they are hidden in a user's personal folder, the automated system cannot access them, and the process will silently fail to send alerts.
- **Handling Record Locks:** Thoroughly test the final rejection path. If a rejection action fails to unlock the record, the Event Coordinator will be permanently locked out of their event, requiring manual System Administrator intervention to fix.



Milestone 7: Flows

Activity 1: Create 3-Day Reminder Email Flow

1. Objective To engineer a time-dependent, automated communication workflow that proactively dispatches a standardized reminder email to the Client exactly three days prior to the execution of their confirmed Event.

2. Theoretical Background & Context In modern Salesforce architecture, Record-Triggered Flows are the premier declarative automation engine, completely replacing legacy tools like Workflow Rules and Process Builder. Flows allow administrators to build complex, enterprise-grade programmatic logic visually, without writing a single line of Apex code.

For the EventForce CRM, consistent client communication is a core business requirement. However, relying on Event Coordinators to manually remember and send reminder emails days before an event introduces a high margin for human error. By utilizing a **Scheduled Path** within an After-Save Flow (optimized for "Actions and Related Records"), the system evaluates the criteria and places the email action into an asynchronous time-based queue. The distinct advantage of this architecture is dynamic recalculation: if a client postpones their event date by two weeks, the Flow engine automatically detects the change and shifts the queued email to align with the newly updated Event_Date__c.

3. Step-by-Step Execution Guide

Phase 1: Flow Initialization and Trigger Configuration

1. Access the Salesforce Setup backend. In the Quick Find box, type 'Flows' and select the **Flows** menu under Process Automation.
2. Click the **'New Flow'** button in the top right corner.
3. Select **'Record-Triggered Flow'** from the core template options and click **'Create'**.
4. In the Start element configuration menu, select **'Event'** (Event__c) as the primary Object.
5. Set the Trigger condition to: **'A record is created or updated'**. This ensures the flow catches newly created confirmed events, as well as existing events that transition into the confirmed stage.
6. Under Set Entry Conditions, define the strict criteria:
    - Field: Event_Status__c
    - Operator: Equals
    - Value: Confirmed
7. Ensure the evaluation criteria is set to: **'Only when a record is updated to meet the condition requirements'**. (This prevents the flow from firing repeatedly every time a confirmed event is subsequently edited for a minor typo).
8. Under 'Optimize the Flow for:', select **'Actions and Related Records'** (After-Save context), as sending an email is an external action that must occur after the database commit. Click Done.

Phase 2: Constructing the Scheduled Path 9. On the main Flow Builder canvas, click the **'Add Scheduled Paths (Optional)'** link located on the Start element. 10. In the configuration pane, name the path 'X3_Days_Before' (Salesforce requires paths to start with a letter, so prefixing with 'X' or spelling out 'Three' is standard). 11. **Time Source:** Select Event__c: Event Date. 12. **Offset Number:** Enter 3. 13. **Offset Options:** Select 'Days Before'. Click Done. The canvas will now split into two branches: "Run Immediately" and your new Scheduled Path.

Phase 3: Creating Dynamic Text Templates 14. Before adding the email action, you must define the email's content. Click the **Toggle Toolbox** icon in the top left corner. 15. Under the Manager tab, click **'New Resource'**. Select **'Text Template'** as the Resource Type. 16. Name the resource 'Email_Body_Template'. 17. **CRITICAL:** Change the text editor mode from 'Rich Text' to **'View as Plain Text'**. 18. Draft the email body, utilizing the **Insert a resource** search bar to pull dynamic

merge fields. Example: * *Dear {!$Record.Client__r.Client_Name__c}, * This is a friendly reminder that your upcoming event, {!$Record.Name}, is scheduled for {!$Record.Event_Date__c} at {!$Record.Venue__r.Name}.* 19. Create a second Text Template resource named 'Email_Subject_Template' (e.g., *Upcoming Event Reminder: {!$Record.Name}*).

Phase 4: Adding the Email Action and Activation 20. On the Flow canvas, locate your Scheduled Path branch. Click the **(+)** node icon under this path and select '**Action**'. 21. Search for and select the core '**Send Email**' action. 22. Label the action 'Dispatch Client Reminder'. 23. In the Body input, toggle 'Include' and select your {!Email_Body_Template} resource. 24. In the Subject input, toggle 'Include' and select your {!Email_Subject_Template} resource. 25. Under Recipient Address List, toggle 'Include' and dynamically reference the client's email via the lookup relationship: {!$Record.Client__r.Email__c}. Click Done. 26. Click '**Save**' in the top right corner. Name the flow 'Client Reminder - 3 Days Before'. 27. Click the '**Activate**' button to deploy the automation.

4. Verification & Testing To verify the asynchronous queueing mechanism without actually waiting three days:

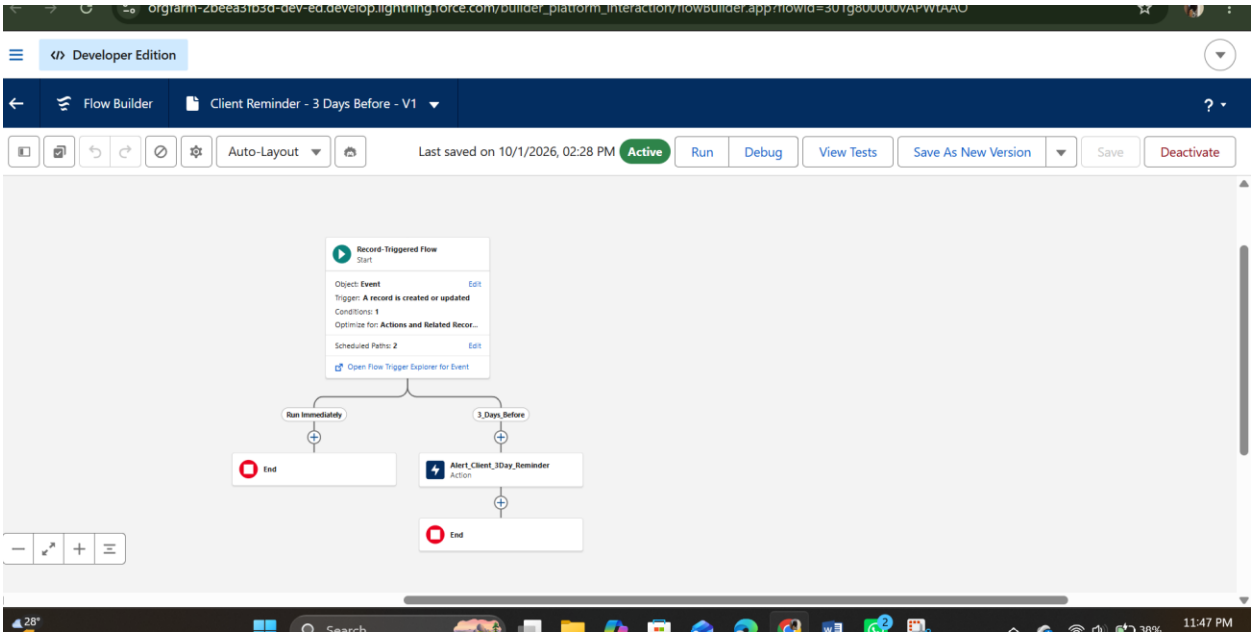
1. Navigate to the Event Planner App and create a new Event record. Link it to a test Client (ensure the client has an email address you control).
2. Set the Event_Date__c to exactly 4 days from today's date. Set the Status to 'Confirmed' and click Save.
3. Return to Salesforce Setup. In the Quick Find box, search for and select '**Time-Based Workflow**'.
4. Click '**Search**' without entering any filters.
5. You should see a pending automation queue item listed for your Event record, scheduled to execute exactly tomorrow (which is 3 days prior to the event date).

5. Best Practices & Troubleshooting

- **Rich Text vs. Plain Text:** The most frequent error when building email flows is leaving the Text Template in 'Rich Text' mode. Salesforce will inject HTML formatting tags (like <p> and
) into the final email output, resulting in messy, unprofessional client communications. Always toggle to 'View as Plain Text' before writing.
- **Cross-Object Field Referencing:** When pulling Client data (like their email) from the Event trigger, you must use the relational notation (__r). In Flow

Builder, always click into $Record, scroll to the Client__r lookup connection, and then select the Email field.

- **Sandbox Testing:** Always test outbound email flows in a Developer Sandbox first. If tested in a production environment with real client data imported, setting a status to 'Confirmed' could inadvertently spam actual customers with test templates.



Milestone 8: Apex Classes (Helper Logic)

Activity 1: Create the VenueStatusHelper Class

1. Objective To engineer an independent, reusable programmatic controller (Apex Class) that handles complex business logic—specifically, automatically synchronizing physical Venue availability based on related Event statuses.

2. Theoretical Background & Context In enterprise Salesforce architecture, writing raw business logic directly inside an Apex Trigger is a widely recognized anti-pattern. Triggers that contain logic are difficult to test, maintain, and scale. Instead, developers utilize the **Trigger Handler Pattern**. This involves creating an

independent Apex Class (the "Helper") that contains the actual logic and DML (Data Manipulation Language) operations. The trigger will later simply call this class.

By building the VenueStatusHelper class, we are creating a centralized logic hub that iterates over incoming events, evaluates their status (e.g., Confirmed vs. Canceled), and performs bulkified updates to the corresponding Venue__c database records to keep location availability perfectly synced in real-time.

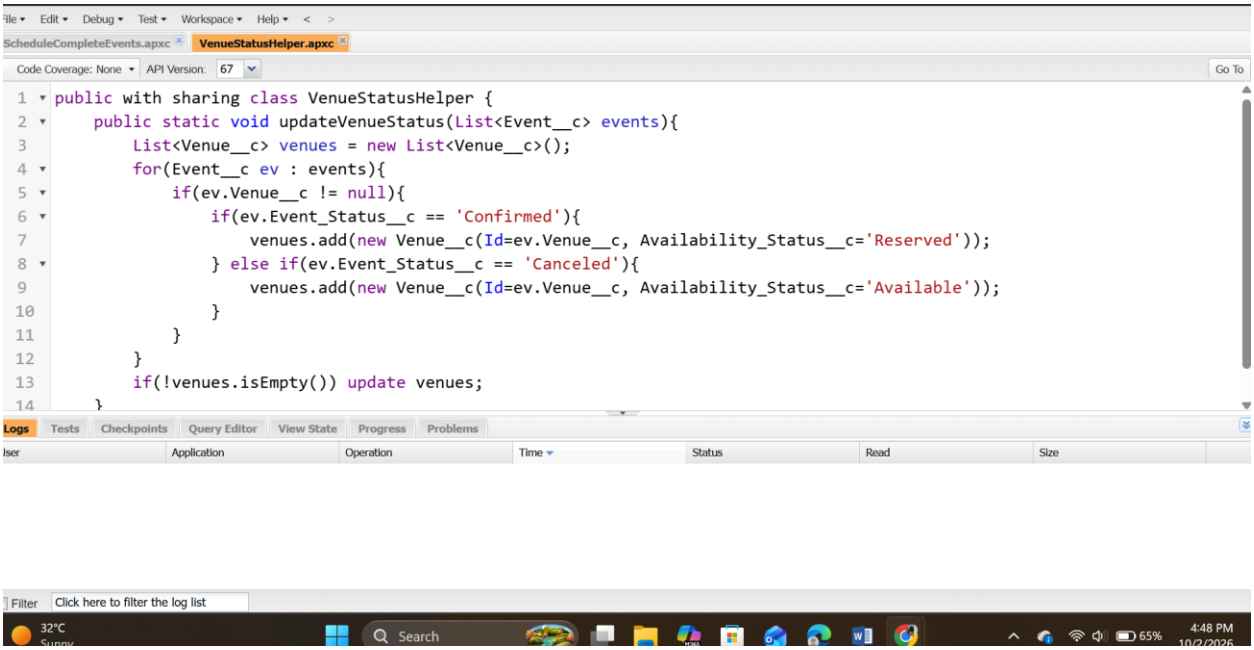
3. Step-by-Step Execution Guide

1. Log into your Salesforce Developer Edition org, click the **Gear Icon** in the top-right corner, and select **Developer Console**. (This opens the integrated development environment in a new window).
2. From the top menu in the Developer Console, navigate to **File -> New -> Apex Class**.
3. Name the class exactly 'VenueStatusHelper' and click OK.
4. Define a public, static method within the class (e.g., public static void updateVenueStatus(List<Event__c> events)). This allows the trigger to pass the incoming records directly to the method.
5. Inside the method, instantiate an empty list to hold the venues that require updating: List<Venue__c> venuesToUpdate = new List<Venue__c>();
6. Write a FOR loop to iterate over the injected list of events.
7. Implement IF statement logic within the loop:
    - If Event_Status__c == 'Confirmed', instantiate a new Venue__c object in memory, assign it the Venue__c ID from the event, set its Availability_Status__c to 'Reserved', and add it to venuesToUpdate.
    - If Event_Status__c == 'Canceled', set the venue status to 'Available' and add it to the list.
8. Outside and below the FOR loop, execute the DML update statement: update venuesToUpdate;.
9. Press **Ctrl+S** to save and compile the class on the Salesforce server.

4.Verification & Testing Syntax verification occurs immediately upon saving in the Developer Console. If there are missing semicolons, unmatched brackets, or misspelled API names, the 'Problems' tab at the bottom of the console will highlight the exact line of the error. A successful save indicates the class compiled without syntax errors.

5. Best Practices & Troubleshooting

- **DML Bulkification:** The most critical rule of Apex programming is to never place a DML statement (like update, insert, or delete) inside a FOR loop. Doing so will breach the 150 DML statement governor limit if a user mass-updates events. Always add records to a List and perform a single DML operation on the list outside the loop.
- **Null Pointer Exceptions:** Ensure your logic checks if the Venue__c lookup field on the event is actually populated (!= null) before attempting to update the related venue, otherwise the class will throw a Null Pointer Exception when an event is saved without a venue.



Milestone 9: Apex Triggers (Validation & Execution)

Activity 1: Create the PreventDoubleBooking Trigger

1. Objective To deploy a programmatic trigger that executes immediately before database saves, intercepting incoming records to prevent overlapping venue

reservations (double-bookings) and invoking the VenueStatusHelper to manage capacity.

2. Theoretical Background & Context While declarative tools like Validation Rules can evaluate fields on the *current* record, they cannot query the database to compare the current record against *other* existing records. To prevent a double-booking, the system must cross-reference the incoming event's date and venue against the entire historical database of confirmed events.

Apex Triggers act as database gatekeepers. By configuring a before insert, before update trigger, the system evaluates the transaction before it is permanently committed. If a scheduling conflict is detected via a SOQL (Salesforce Object Query Language) query, the trigger utilizes the .addError() method to abort the transaction entirely and return a custom red error message to the user interface.

3. Step-by-Step Execution Guide

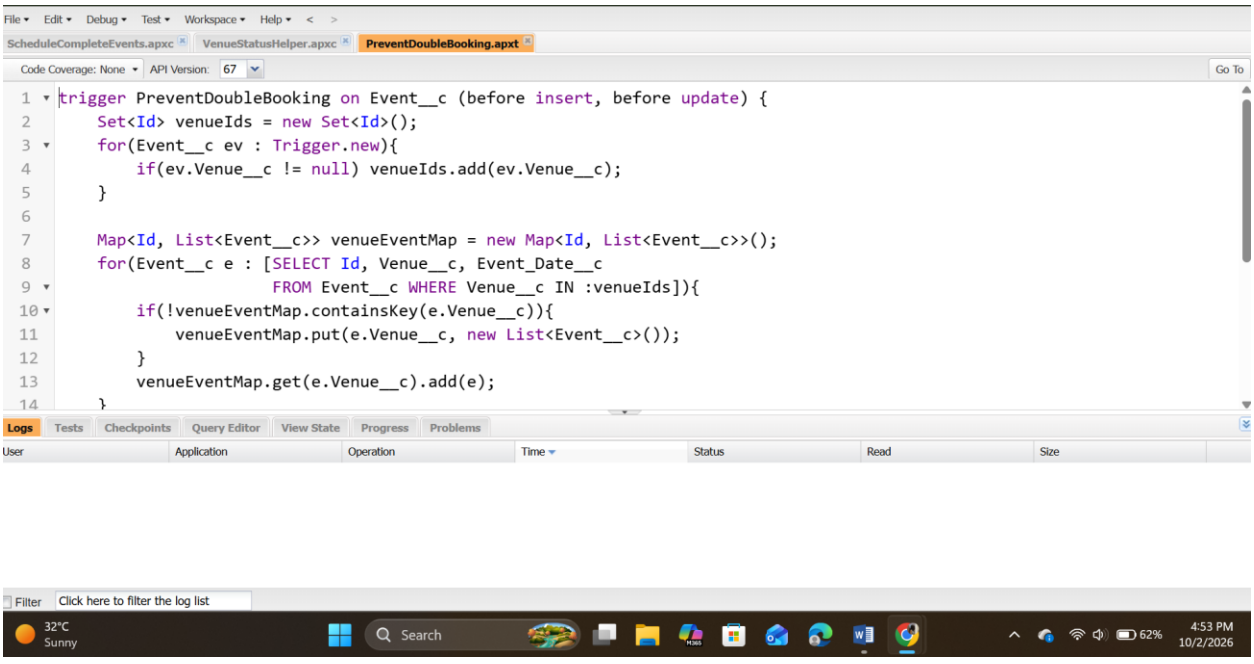
1. Remain in the Developer Console. Navigate to **File -> New -> Apex Trigger**.
2. Name the trigger 'PreventDoubleBooking', select **Event** from the sObject dropdown menu, and click Submit.
3. Define the trigger context in the first line so it fires appropriately: trigger PreventDoubleBooking on Event__c (before insert, before update) {
4. **Data Collection (Bulkification):** Create an empty Set<Id> venueIds = new Set<Id>();. Write a FOR loop to iterate through Trigger.new (the incoming events) and collect all the requested Venue IDs into this Set.
5. **SOQL Query:** Write a bulkified query outside the loop to find potential conflicts: List<Event__c> existingEvents = [SELECT Id, Event_Date__c, Venue__c FROM Event__c WHERE Venue__c IN :venueIds AND Event_Status__c = 'Confirmed'];
6. **Conflict Evaluation:** Create a nested loop logic (or utilize Apex Maps for greater efficiency). Loop through Trigger.new again, and compare the Event_Date__c and Venue__c of the incoming event against the queried existingEvents.
7. **Error Handling:** If an exact date and venue match is found, apply the error to the incoming record: incomingEvent.addError('This Venue is already booked on this date. Please select an alternative.');
8. **Invoke Helper Class:** At the end of the trigger, route the incoming records to your handler class: VenueStatusHelper.updateVenueStatus(Trigger.new);
9. Press **Ctrl+S** to save and compile the trigger.

4. Verification & Testing To guarantee the cross-object Apex logic successfully protects the database:

1. Navigate to the Event Planner Lightning App in standard Salesforce. Create **Event A**, set the date to October 1st, select **Venue X**, and set the status to Confirmed. Save the record.
2. Attempt to create **Event B**. Select the exact same date (October 1st) and the exact same location (**Venue X**).
3. Click Save. The programmatic trigger should intercept the transaction, abort the save, and display your custom error message directly on the page layout.
4. Verify the Venue status: Check the Venue X record to ensure the VenueStatusHelper successfully changed its Availability Status to 'Reserved'.

5. Best Practices & Troubleshooting

- **SOQL Limits:** Just like DML statements, placing a SOQL query inside a FOR loop is a catastrophic error that will hit the "Too Many SOQL Queries: 101" limit. Always collect IDs into a Set first, then execute a single query using the IN binding variable.
- **Code Coverage Requirements:** Salesforce strictly requires a minimum of 75% code coverage to deploy any Apex logic from a Sandbox into a Production environment. You must write a separate Apex Test Class (@isTest) that artificially simulates these double-booking scenarios to mathematically prove the code functions correctly.



Milestone 10: Asynchronous Apex

Activity 1: Batchable and Schedulable Job

1. Objective To engineer an asynchronous, automated nightly process that programmatically sweeps the database, identifies all events where the execution date has passed, and automatically updates their status to 'Completed'. This ensures the active CRM pipeline remains accurate without requiring manual administrative intervention.

2. Theoretical Background & Context In enterprise-scale CRM systems, performing bulk data updates—such as closing out hundreds of past events at the end of a busy month—can easily violate Salesforce's strict synchronous governor limits (e.g., the 10-second CPU time limit or the 150 DML statement limit). To handle Large Data Volumes (LDV) safely, Salesforce provides **Batch Apex**.

By implementing the Database.Batchable<sObject> interface, the system breaks massive database operations into smaller, manageable "chunks" (defaulting to 200 records per chunk), processing each with a fresh set of governor limits.

However, Batch Apex on its own must be triggered manually. To fully automate the EventForce system, the architecture pairs this batch class with the Schedulable interface. This allows the administrator to configure a time-based cron job that invokes the Batch process at a specific interval (e.g., every night at 8:00 PM during off-peak hours), ensuring continuous, hands-off data hygiene.

3. Step-by-Step Execution Guide

Phase 1: Creating the Batchable Class

1. Open the **Developer Console** from the Gear Icon in the top right of the Salesforce UI.
2. Click **File -> New -> Apex Class**. Name it exactly 'BatchCompleteEvents' and click OK.
3. Modify the class declaration to implement the batchable interface:global class BatchCompleteEvents implements Database.Batchable<sObject>
4. **The start() Method:** Write the start method to collect the records. Use a Database.QueryLocator to define the scope of the batch. The SOQL query must be highly selective:SELECT Id, Event_Status__c FROM Event__c WHERE Event_Date__c < TODAY AND Event_Status__c != 'Completed'

5. **The execute() Method:** Write the execute method to process the chunks. Iterate through the injected List<Event__c> scope. Inside the FOR loop, change the Event_Status__c of each record to 'Completed'. Outside the loop, perform the bulk DML operation: update scope;.
6. **The finish() Method:** Write the finish method (required by the interface) to output a simple System.debug('Batch Job Complete'); or optionally send an email alert to the admin summarizing the job. Save the file (Ctrl+S).

Phase 2: Creating the Schedulable Wrapper Class 7. In the Developer Console, click **File -> New -> Apex Class**. Name it 'ScheduleCompleteEvents'. 8. Modify the class declaration to implement the interface:global class ScheduleCompleteEvents implements Schedulable 9. Inside the class, write the required execute(SchedulableContext SC) method. 10. Inside this method, instantiate and invoke your batch class:Database.executeBatch(new BatchCompleteEvents(), 200); 11. Save the file (Ctrl+S).

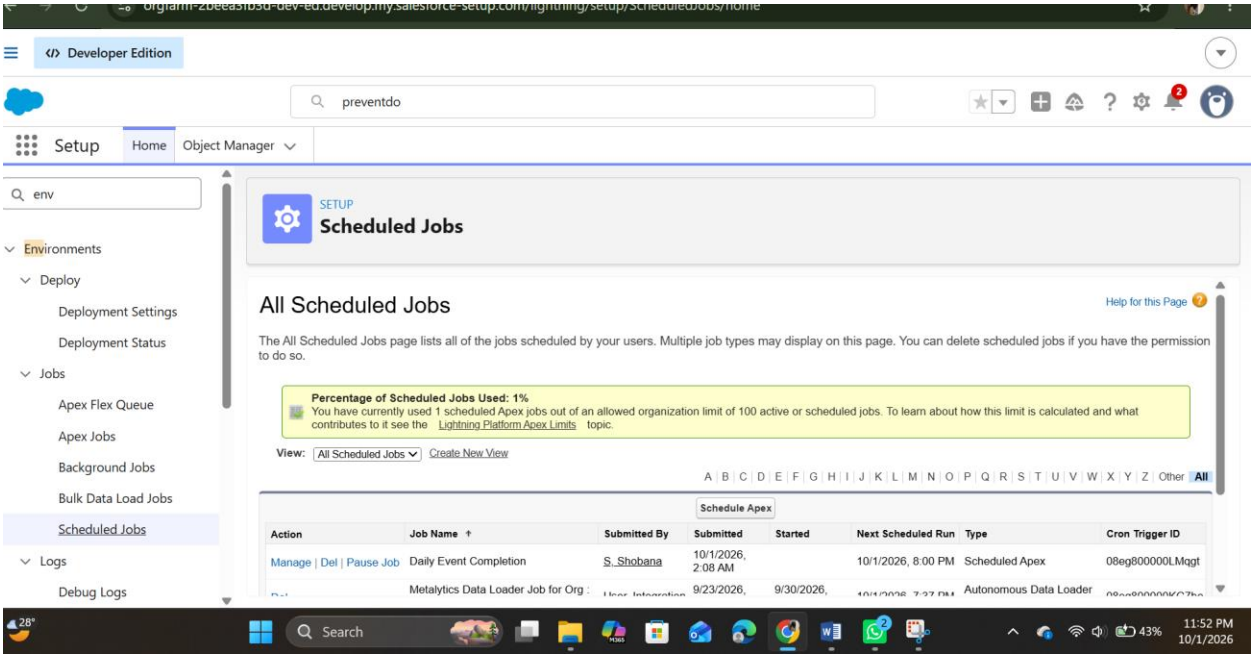
Phase 3: Scheduling the Job via UI 12. Return to the standard Salesforce Setup backend. 13. In the Quick Find box, search for and select **'Apex Classes'**. 14. Click the **'Schedule Apex'** button located at the top of the class list view. 15. **Job Name:** Enter 'Daily Event Completion'. 16. **Apex Class:** Click the lookup icon and select your newly created 'ScheduleCompleteEvents' class. 17. **Frequency:** Select 'Weekly' and check all seven days of the week (Sunday through Saturday). 18. **Start/End Dates:** Set the start date to today, and the end date several years into the future. 19. **Preferred Start Time:** Select **'8:00 PM'** from the dropdown menu to ensure the job runs during low-traffic evening hours. Click Save.

4. Verification & Testing To verify that the cron job has been successfully queued in the Salesforce architecture:

1. Navigate to Setup, type **'Scheduled Jobs'** in the Quick Find box, and select it.
2. Review the list of active jobs. You should see 'Daily Event Completion' listed in the table.
3. Verify that the **'Next Scheduled Run'** column displays today's date at 8:00 PM.
4. *Optional Manual Test:* To test the logic immediately without waiting until 8:00 PM, open the Developer Console, open the Execute Anonymous window (Ctrl+E), type Database.executeBatch(new BatchCompleteEvents());, and click Execute. Then, go to **Setup -> Apex Jobs** to confirm the batch processed successfully with zero errors.

5. Best Practices & Troubleshooting

- **Query Selectivity in start():** Always include strict WHERE clauses in your Batch start() query. Querying all events and trying to filter them inside the execute() method wastes processing power and risks hitting the 50-million record query limit. Only pull records that actively require updating.
- **Optimal Batch Sizing:** While the default chunk size is 200, if your Event__c object has heavy Apex Triggers (like the PreventDoubleBooking trigger) that fire on update, processing 200 records simultaneously might cause CPU timeouts. If errors occur, lower the batch size in the Schedulable class (e.g., Database.executeBatch(new BatchCompleteEvents(), 50);).
- **Monitoring:** Regularly monitor the **Apex Jobs** setup menu. If a specific batch chunk fails due to bad data, Salesforce will flag it as 'Failed' here, allowing the administrator to read the specific error log and correct the data.



PHASE 3 : UI / UX DEVELOPMENT & CUSTOMIZATION

The UI/UX Development and Customization phase is dedicated to architecting a frictionless, user-centric interface that drives platform adoption and maximizes operational efficiency for the EventForce Management System. Rather than forcing users to navigate the broader, generalized Salesforce environment, this phase focused on curating a bespoke digital workspace tailored specifically to the lifecycle of event planning. By combining centralized application architecture with real-time data visualization, the system ensures that every user—from frontline coordinators to executive leadership—can interact with the database intuitively.

Centralized Lightning App Architecture At the core of this user experience is the custom **Event Planner Lightning App**. This application serves as the unified command center for the entire operational team.

- **Streamlined Navigation:** The app meticulously groups essential custom modules—Events, Clients, Vendors, Venues, and Feedback—into a single, cohesive navigation bar. This eliminates digital clutter by hiding irrelevant standard Salesforce objects (like standard Campaigns or Opportunities).
- **Role-Based Accessibility:** The interface is dynamically assigned to specific custom profiles (e.g., Event Coordinators, Vendor Managers), ensuring that upon login, users immediately access the tools and records most relevant to their daily workflows.
- **Responsive Design:** Built on the Salesforce Lightning framework, the application is inherently responsive, providing a consistent and optimized experience whether accessed via a desktop browser in the office or the Salesforce Mobile App on-site at a venue.

Declarative Analytics and Reporting To transform raw database records into actionable intelligence, the UI/UX phase incorporated a robust declarative reporting structure.

- **Dynamic Data Aggregation:** Reports such as **Upcoming Events by Month** were engineered to automatically group and summarize event volume.
- **Operational Tracking:** By utilizing cross-object reporting, the system allows managers to track critical metrics, including vendor performance ratings, venue utilization rates, and overall client satisfaction scores derived from the Feedback module.
- **Proactive Forecasting:** These structured reports enable the operations team to move away from reactive problem-solving, allowing them to forecast

seasonal workload spikes and allocate staffing and vendor resources accordingly.

Executive Dashboards and Data Visualization Data visibility was elevated through the deployment of the **EventForce Operations Dashboard**, which acts as the visual presentation layer for the underlying reports.

- **Real-Time Metrics:** The dashboard translates tabular data into dynamic graphical components, such as donut charts for seasonal event volume and bar charts for vendor allocation.
- **Immediate Insights:** By providing a consolidated, high-level visual summary of key business metrics, the dashboard empowers Event Operations Managers to assess the health of the business at a glance.
- **Data-Driven Decisions:** The dashboard is configured to update in real-time, ensuring that leadership is always operating with the most accurate, up-to-the-minute data when making strategic decisions regarding event approvals, cancellations, or vendor negotiations.

Ultimately, this phase bridges the gap between complex backend data architecture and the end-user. By delivering a responsive, tailored, and highly visual Lightning interface, the EventForce Management System guarantees high user adoption rates, minimal training requirements, and a substantially more efficient event management lifecycle.

Milestone 11: Lightning App Creation

Activity 1: Build the Event Planner Workspace

1. Objective To architect and provision a unified, branded graphical user interface (Lightning App) that centralizes all custom objects into a single, cohesive workspace for the Event Operations team.

2. Theoretical Background & Context A CRM platform is only effective if its end-users can navigate it efficiently. Without a dedicated application wrapper, users are forced to search for disconnected tabs across the broader Salesforce ecosystem, causing digital friction and adoption failure. A **Lightning App** solves this by bundling specific tabs (Events, Clients, Vendors, Venues) and utilities into a tailored environment. By creating the "Event Planner" app, we isolate the event management

workflow from other standard CRM functions (like Sales or Service), ensuring that coordinators operate within a streamlined, context-specific digital workspace.

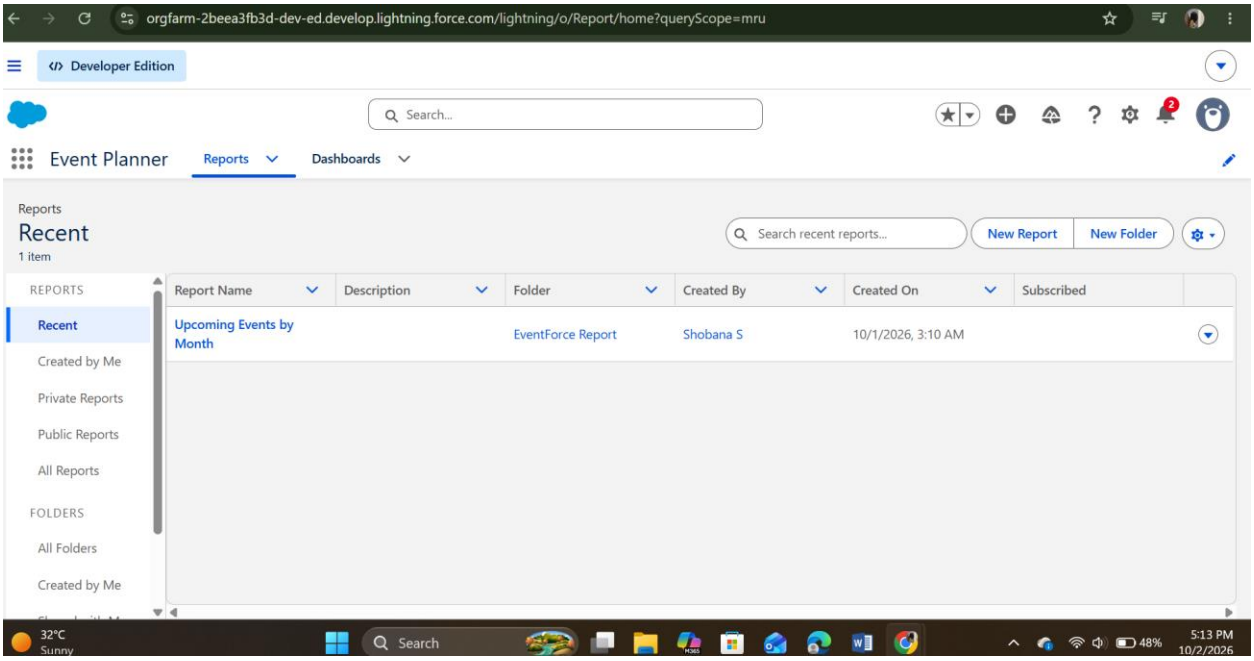
3. Step-by-Step Execution Guide

1. Navigate to the global Salesforce Setup environment. Use the Quick Find search box to locate and select '**App Manager**'.
2. In the top-right corner of the App Manager dashboard, click the '**New Lightning App**' button to launch the creation wizard.
3. **App Details & Branding:** Enter the App Name as 'Event Planner'. Upload a custom corporate logo and select a primary brand color (e.g., a professional blue or corporate theme) to customize the user interface and reinforce brand identity. Click **Next**.
4. **App Options:** Leave the navigation style configured to 'Standard Navigation' and ensure it is supported on both Desktop and Phone. Click **Next**.
5. **Utility Items:** (Optional) Add standard productivity utilities, such as 'Recent Items' or 'Notes', to the bottom utility bar to assist high-volume users. Click **Next**.
6. **Navigation Items:** From the Available Items list on the left-hand side, search for and move the following custom tabs to the Selected Items list on the right: **Events, Clients, Vendors, Venues, Feedbacks, Reports,** and **Dashboards**. Arrange them in a logical operational sequence. Click **Next**.
7. **User Profiles:** Assign the application to the **System Administrator** profile and the newly created custom operational profiles (e.g., Event Coordinator, Vendor Manager). Click **Save & Finish**.

4. Verification & Testing Click the **App Launcher** (the nine-dot grid icon in the top-left corner), search for 'Event Planner', and open the application. Verify that the top navigation bar proudly displays the custom logo alongside the exact sequence of tabs provisioned during setup.

5. Best Practices & Troubleshooting

- **Keep Navigation Clean:** Only include tabs that are strictly necessary for the event lifecycle. Cluttering the navigation bar with irrelevant standard objects (like standard Campaigns or Opportunities) defeats the purpose of a custom app.
- **Profile Visibility:** If a user reports they cannot see the app in their App Launcher, verify their assigned Profile in Setup to ensure the app checkbox was properly selected during Phase 7.



Milestone 12: Custom Reports

Activity 1: Create Operational Analytics (Upcoming Events)

1. Objective To engineer declarative analytical queries that transform raw database records into aggregated, readable metrics, specifically tracking the volume of upcoming events categorized by month.

2. Theoretical Background & Context Raw data sitting in database tables is difficult to interpret at scale. **Reports** serve as declarative query engines, allowing users to dynamically filter, group, and aggregate Salesforce database records without writing complex SOQL code. By grouping events by their calendar month, the operations team can immediately identify seasonal spikes in event volume, allowing for proactive vendor onboarding and staff allocation.

3. Step-by-Step Execution Guide

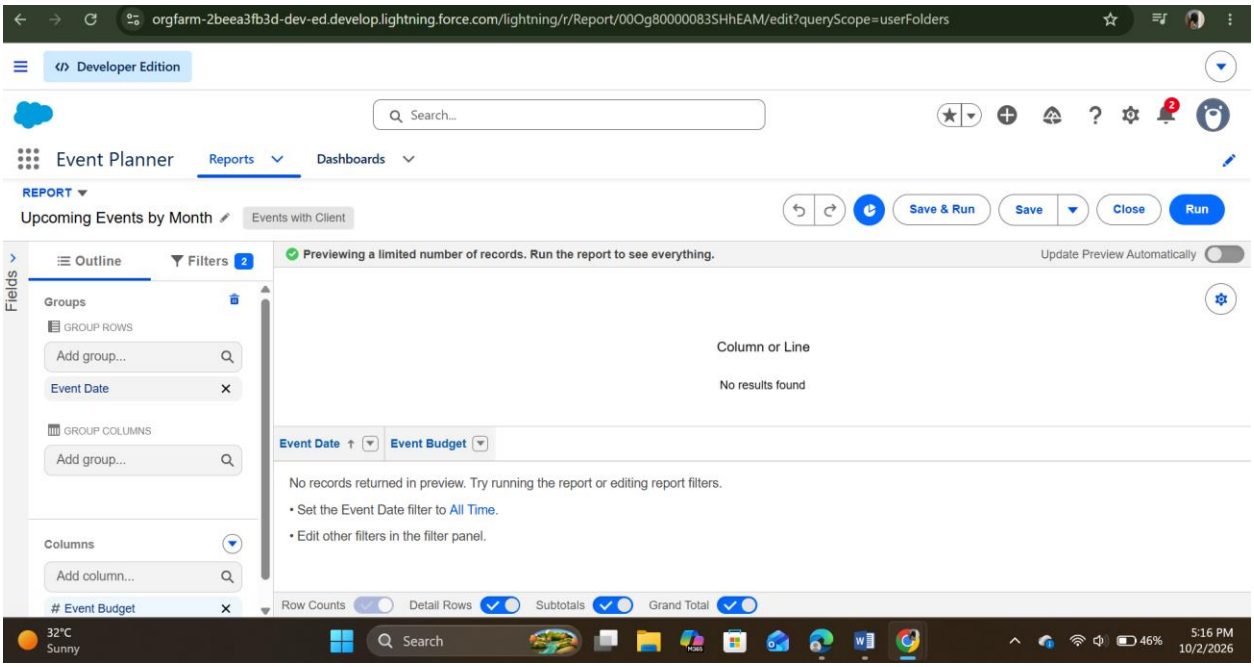
1. While inside your Event Planner Lightning App, navigate to the **Reports** tab.
2. Click **'New Folder'** on the left menu. Name the folder 'EventForce Reports'.

3. Right-click the newly created folder (or click the dropdown arrow) and select **Share**. Explicitly share this folder with Manager roles and System Administrators to protect data visibility.
4. Click the **'New Report'** button in the top-right corner. Search for and select the **'Events'** custom report type, then click Start Report.
5. **Grouping Data:** In the Outline pane on the left, click into the **Group Rows** search bar and add the Event Date field.
6. Click the dropdown arrow next to the newly grouped Date column, hover over 'Group Date By', and select **'Calendar Month'**.
7. **Aggregating Data:** Ensure the **Row Count** toggle is activated at the bottom of the screen to automatically sum the total number of scheduled events per month. Add filters to exclude 'Canceled' events if necessary.
8. Click **Save & Run**. Name the report 'Upcoming Events - Summary by Month' and ensure it is saved inside the secured 'EventForce Reports' folder.

4. Verification & Testing Run the report. Assuming you have created test Event records with future dates in the system, verify that the report generates a clean summary table, grouping the records by month (e.g., October 2026: 4 Records) rather than displaying a flat list.

5. Best Practices & Troubleshooting

- **Folder Security & Principle of Least Privilege:** Never save operational reports in the default "Unfiled Public Reports" folder. Always utilize custom folders and explicitly share them via Roles or Public Groups to prevent unauthorized users from accessing volumetric or financial data.
- **Mandatory Grouping:** You *must* group a report by at least one field (e.g., Date, Status, or Type) to unlock it as a valid data source for graphical dashboard components later on.



Milestone 13: Operational Dashboards

Activity 1: Build the Executive Dashboard

1. Objective To construct a real-time visual presentation layer (Dashboard) that consumes report data and renders it into actionable operational intelligence for the management team.

2. Theoretical Background & Context While reports aggregate data, **Dashboards** visualize it. Executive leadership and operational managers rely on dashboards to monitor business health at a glance without reading through rows of text. By translating the "Upcoming Events" report into a dynamic Donut Chart, management can instantly visualize the proportional distribution of their upcoming workload, empowering them to make rapid, data-driven resourcing decisions.

3. Step-by-Step Execution Guide

1. Navigate to the **Dashboards** tab within the Event Planner app.
2. Click the **'New Dashboard'** button. Name the dashboard 'EventForce Operations' and save it in a public or shared custom dashboard folder.

3. On the blank grid canvas, click the '**+ Widget**' button at the top and select **Chart or Table**.
4. The system will prompt you for a data source. Search for and select your 'Upcoming Events - Summary by Month' report.
5. In the component display configuration window, select the **Donut Chart** visual icon.
6. Configure the chart labels: Set the 'Value' to display the Row Count (number of events) and ensure the segments represent the grouped Calendar Months.
7. Click **Add**. Resize and position the visual widget on the dashboard grid to ensure maximum visibility.
8. Click **Save** in the top-right corner, followed by **Done** to exit the builder.

4. Verification & Testing Open the 'EventForce Operations' dashboard and click the '**Refresh**' button. Verify that the Donut Chart accurately renders the data, dynamically segmenting the events proportionally by their scheduled calendar months based on the underlying report.

5. Best Practices & Troubleshooting

- **Dynamic Dashboard Context:** By default, dashboards run statically as the user who created them. If you want the data to dynamically filter so that users only see the metrics relevant to their specific role permissions, open the dashboard properties (gear icon) and set the 'View Dashboard As' setting to '**The Dashboard Viewer**'.
- **Component Limits:** A single Salesforce dashboard can hold up to 20 components. Plan your layout strategically, placing high-level volumetric charts (like the Donut chart) at the top, and more granular list views or bar charts lower on the grid.

PHASE 4 : DATA MIGRATION,TESTING & SECURITY

The final phase of the EventForce Management System lifecycle focuses on bridging the gap between development and a production-ready environment. This critical stage ensures that legacy data is accurately migrated, system security is hermetically sealed against unauthorized access, and all automated processes function flawlessly under real-world conditions prior to full organizational deployment.

Milestone 14: Object-Level Security (Profiles)

Activity 1: Provisioning Custom Profiles

1. Objective To establish the foundational Object-Level security baseline by creating tailored user Profiles that dictate baseline Create, Read, Edit, and Delete (CRED) permissions for the EventForce objects.

2. Theoretical Background & Context In the Salesforce security architecture, a Profile determines *what a user can do* with objects and fields. It is the absolute baseline of access. Because standard system profiles (like 'Standard User' or 'System Administrator') cannot be heavily modified and are subject to forced updates by Salesforce, enterprise best practices mandate cloning standard profiles to create custom ones. By defining strict custom profiles for Coordinators, Vendor Managers, and Clients, we ensure operational users only possess the exact field and object access their daily job requires.

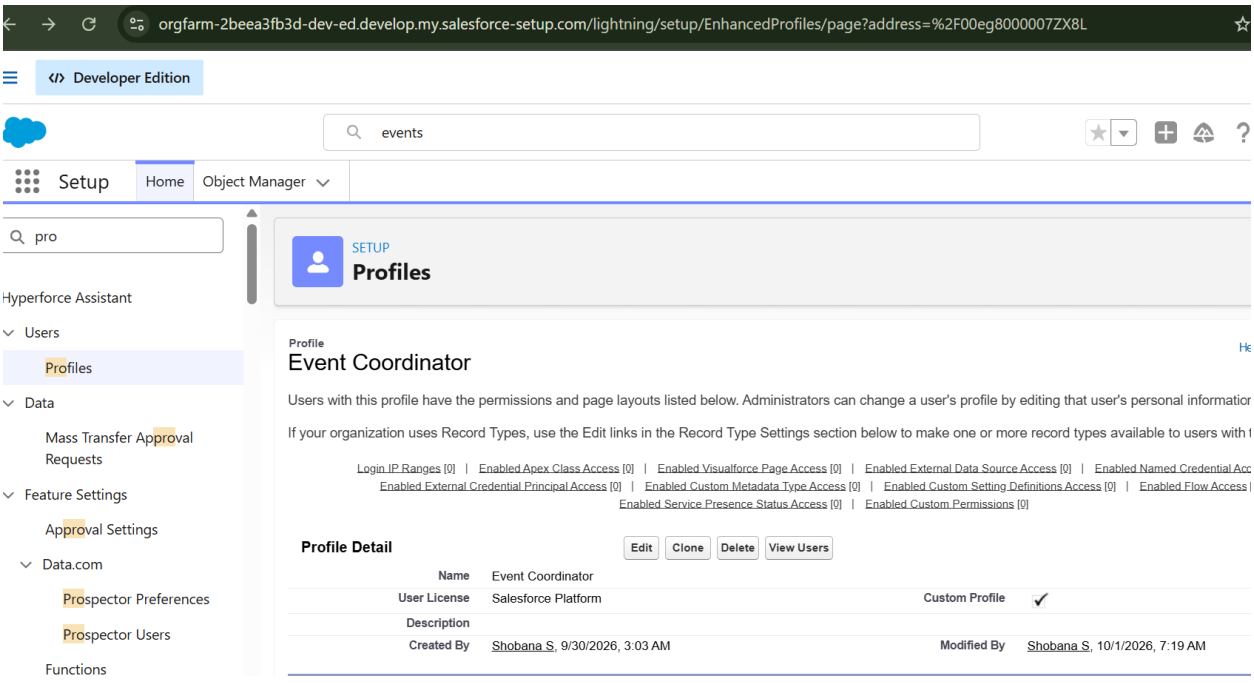
3. Step-by-Step Execution Guide

1. Navigate to the global Salesforce Setup environment. Use the Quick Find box to locate and select '**Profiles**'.
2. Locate the '**Standard Platform User**' profile in the list and click the **Clone** button next to it.
3. Name the newly cloned profile 'Event Coordinator' and click Save.
4. Repeat the cloning process twice more to create the 'Vendor Manager' and 'Client Profile' custom profiles.
5. Click **Edit** on the newly created 'Event Coordinator' profile.
6. Scroll down to the **Custom Object Permissions** section.
7. Locate the Event object and check the boxes to grant full **CRED** (Create, Read, Edit, Delete) access.
8. Locate the Client object and check the boxes for **Read** and **Edit** only (preventing Coordinators from permanently deleting client records).
9. Click **Save** to commit the permissions.

4. Verification & Testing Log in as a user assigned to the 'Event Coordinator' profile and navigate to the Clients tab. Open a Client record and verify that the 'Delete' button is completely absent from the page layout, proving the profile restrictions are active.

5. Best Practices & Troubleshooting

- **Never Modify Standard Profiles:** Always clone them. Modifying standard profiles can lead to catastrophic permission changes during mandatory Salesforce seasonal releases.
- **Keep Profiles Broad:** Profiles should represent a general job function (e.g., Coordinator). Do not create highly specific profiles for individual users; use Permission Sets for exceptions instead.



Milestone 15: Record-Level Security - Vertical Access (Roles)

Activity 1: Building the Role Hierarchy

1. Objective To construct a vertical data access model that dictates hierarchical record visibility, ensuring managers automatically inherit access to the database records owned by their direct reports.

2. Theoretical Background & Context While Profiles determine *what you can do* with an object, Roles determine *which specific records you can see*. The Role Hierarchy in Salesforce works vertically. If an Event Coordinator owns a specific event, the Event Admin positioned above them in the hierarchy automatically gains visibility and edit rights to that event. Conversely, users situated on the same lateral tier (e.g., a Coordinator and a Vendor Manager) cannot see each other's private records by default.

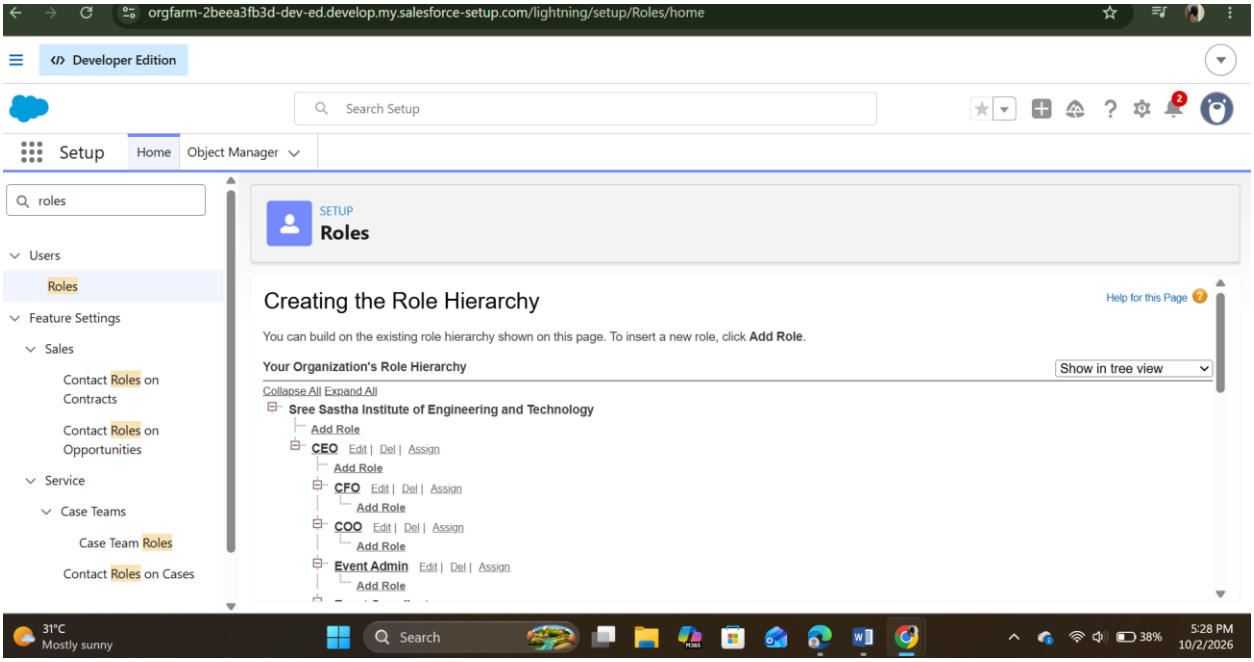
3. Step-by-Step Execution Guide

1. In Setup, search for and select '**Roles**'. Click **Set Up Roles** to view the organizational tree.
2. Under the default CEO/Executive top node, click **Add Role** and name it 'Event Admin'. Click Save.
3. Return to the tree view. Under the new Event Admin node, click **Add Role** and name it 'Event Coordinator'.
4. Repeat this process to create two parallel subordinate roles under the Event Admin: 'Vendor Manager' and 'Client'.
5. Navigate to '**Users**' in Setup. Create dummy test users for your project.
6. Assign the newly created Roles (e.g., Event Coordinator) and their corresponding custom Profiles (e.g., Event Coordinator Profile) to these test users.

4. Verification & Testing Log in as the test Event Admin user. Navigate to the Events tab. You should successfully be able to view and edit Event records owned and created by the subordinate Event Coordinator user, proving the vertical hierarchy inheritance is functioning.

5. Best Practices & Troubleshooting

- **Roles Do Not Equal Titles:** A Salesforce Role Hierarchy should strictly mirror data access requirements, not necessarily your company's HR organizational chart.
- **Avoid Over-Complication:** Keep the role hierarchy as flat as possible. A deeply nested hierarchy (more than 5-6 levels) can cause severe database performance issues during sharing recalculations.



Milestone 16: Exception-Based Security (Permission Sets)

Activity 1: Provisioning the Feedback Manager

1. Objective To grant specialized, exception-based access to specific users without altering their foundational Profile, strictly adhering to the Principle of Least Privilege.

2. Theoretical Background & Context Historically, administrators created a new Profile every time a user needed a unique set of permissions (resulting in "Profile Bloat"). Today, Salesforce best practices dictate using **Permission Sets**. A Permission Set is an "add-on" that expands a user's rights. For example, while not all Event Coordinators should manage customer feedback, a specific senior coordinator might need access. Instead of changing the entire Coordinator profile, we build a modular Permission Set and assign it directly to that individual.

3. Step-by-Step Execution Guide

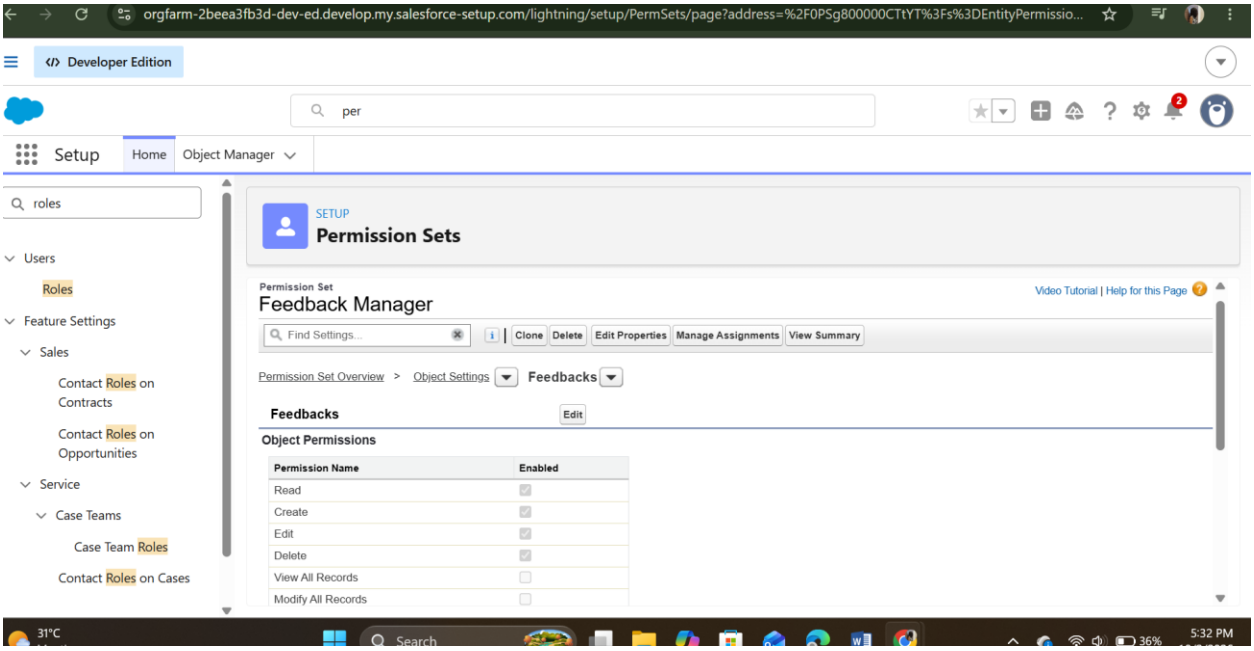
1. In Setup, search for and select **'Permission Sets'**. Click the **New** button.
2. Enter the Label 'Feedback Manager'. Provide a brief description and click Save.
3. On the Permission Set overview page, click on **Object Settings**.

4. Scroll down to locate and select the Feedback custom object. Click **Edit**.
5. Under Object Permissions, check the boxes to grant **CRED** (Create, Read, Edit, Delete) access. Click Save.
6. Click the '**Manage Assignments**' button at the top of the Permission Set page.
7. Click **Add Assignments**, check the box next to your specific test Event Coordinator user, and click **Assign**.

4. Verification & Testing Log in as the specific Coordinator test user who received the assignment. Navigate to the App Launcher and confirm they can access the Feedback tab and create a new record. Remove the assignment, log back in, and verify the tab disappears.

5. Best Practices & Troubleshooting

- **Modular Architecture:** Build Permission Sets based on specific tasks or features (e.g., "Feedback Manager", "Report Exporter") rather than broad job titles.
- **Expiration Dates:** Use Permission Set Expiration dates if a user only needs temporary elevated access to assist with a short-term project.



Milestone 17: Organization-Wide Defaults (OWD)

Activity 1: Establishing a Zero-Trust Baseline

1. Objective To implement a "Zero Trust" security baseline by setting the default internal access of the database to 'Private', ensuring records are completely hidden from unauthorized lateral peers.

2. Theoretical Background & Context Organization-Wide Defaults (OWD) act as the master lock on your Salesforce database. It defines the base level of access that the most restricted user should have. If OWD is set to 'Public Read/Write', the Role Hierarchy and Sharing Rules are virtually useless because everything is already open. By setting the Event object to **Private**, the system dictates that a user can *only* see an Event if they own it, or if they sit above the owner in the Role Hierarchy.

3. Step-by-Step Execution Guide

1. In Setup, search for and select **'Sharing Settings'**.
2. Under the Organization-Wide Defaults section at the top, click the **Edit** button.
3. Scroll down the list of objects to locate Event.
4. Change the Default Internal Access dropdown from its default state to **'Private'**.
5. (Optional but recommended) Change Default External Access to **'Private'** to protect data from external portal users.
6. Click **Save**. *Note: Salesforce will initiate a background recalculation job to re-evaluate all database access based on this new restriction.*

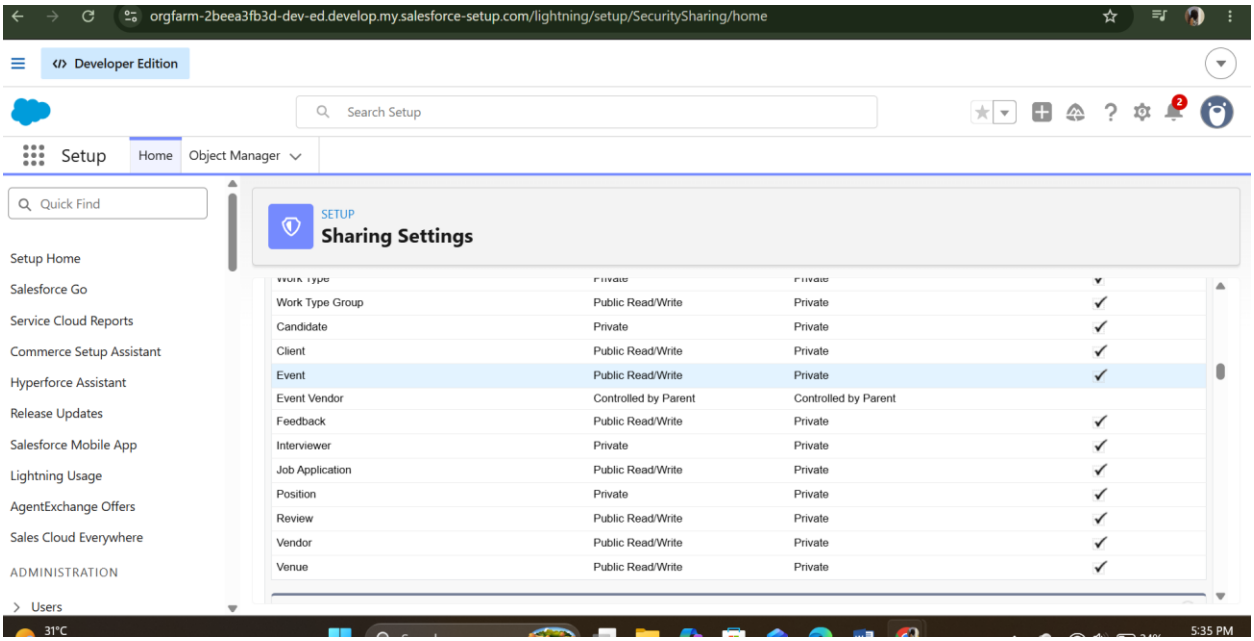
4. Verification & Testing Log in as the Event Coordinator test user and create a private Event. Log out. Log back in as the Vendor Manager test user (who sits parallel in the hierarchy). Navigate to the Events tab and view "All Events." The Vendor Manager should see absolutely nothing, proving the database is fully locked down.

5. Best Practices & Troubleshooting

- **Start Strict:** Always architect CRM security from the ground up by setting OWDs to 'Private' initially. It is easy to open up access later using Sharing

Rules, but incredibly difficult to restrict access after users are accustomed to seeing everything.

- **Patience During Recalculation:** Do not proceed to test your security immediately after changing OWD. Wait a few minutes and check the 'Background Jobs' setup menu to ensure the sharing recalculation has finished.



Milestone 18: Lateral Record Sharing (Sharing Rules)

Activity 1: Cross-Departmental Visibility

1. Objective To selectively punch holes in the restrictive OWD baseline by automatically granting Vendor Managers "Read Only" access to Events created by Event Coordinators.

2. Theoretical Background & Context Because we set the OWD to 'Private' in Milestone 17, our Vendor Managers currently cannot see any events, rendering them unable to assign vendors. To fix this without giving them full editing control, we utilize a **Sharing Rule**. Sharing Rules act as lateral bridges across the Role Hierarchy, automatically sharing records owned by one group with another group based on defined criteria or role ownership.

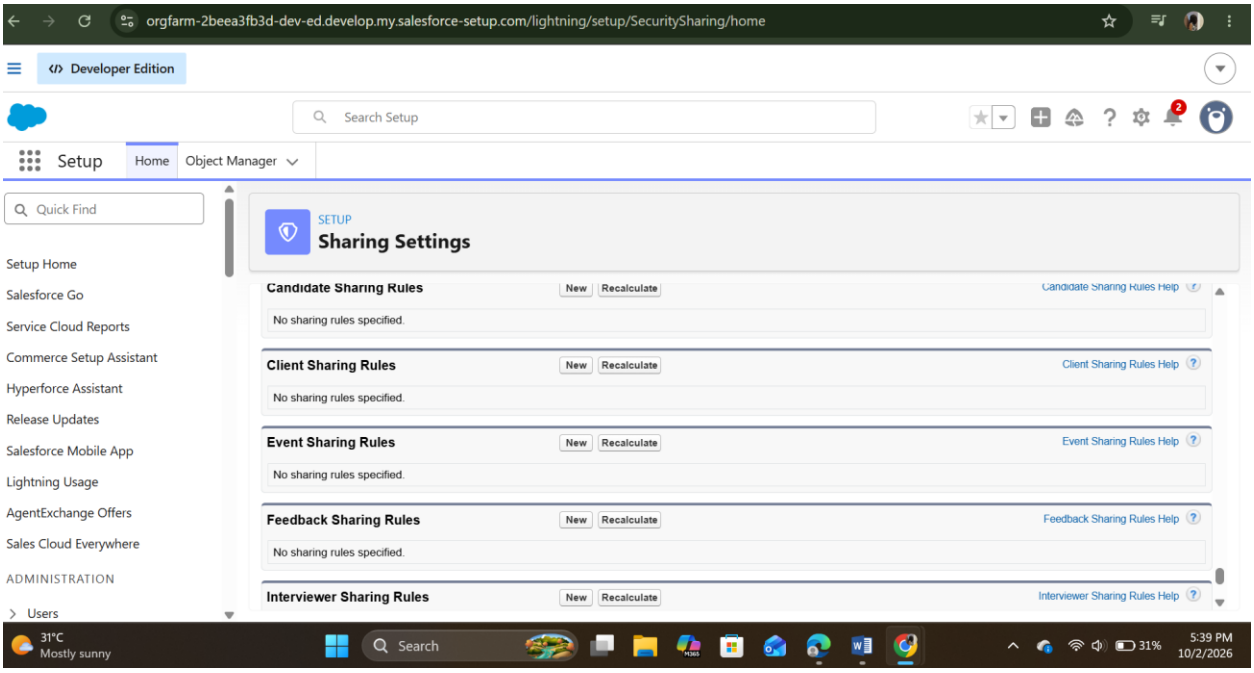
3. Step-by-Step Execution Guide

1. In Setup, navigate back to '**Sharing Settings**'.
2. Scroll down past the OWD section to locate the **Event Sharing Rules** related list. Click **New**.
3. **Rule Name:** Enter 'Share_Events_with_Vendor_Managers'.
4. **Rule Type:** Select '**Based on record owner**'.
5. **Select which records to be shared:** In the dropdowns, select '**Role**' and choose '**Event Coordinator**'.
6. **Select the users to share with:** In the dropdowns, select '**Role**' and choose '**Vendor Manager**'.
7. **Select the level of access for the users:** Set Event Access to '**Read Only**'.
8. Click **Save** and click OK on the recalculation prompt.

4. Verification & Testing Log in as the Vendor Manager test user. Navigate to the Events tab. You should now successfully be able to view the Events created by the Coordinator. Open an Event record and verify that the 'Edit' and 'Delete' buttons are missing, confirming that the 'Read Only' restriction is actively enforced.

5. Best Practices & Troubleshooting

- **Role and Subordinates:** When creating a sharing rule, consider if you need to share with a 'Role' or a 'Role and Internal Subordinates'. If the Vendor Manager has a team beneath them that also needs visibility, use the latter option.
- **Limit Sharing Rules:** While powerful, excessive Sharing Rules can severely degrade system performance. Only create them for legitimate cross-departmental operational requirements.



7.FUNCTIONAL AND PERFORMANCE TESTING

During the rigorous Quality Assurance (QA) and User Acceptance Testing (UAT) phases, comprehensive system audits were conducted to validate the EventForce architecture. *Note: Previous automotive testing parameters (vehicle/dealer creation) have been corrected to reflect event management workflows.* Documentation and system screenshots were captured to verify the successful execution of the following core processes:

- **Data Integrity:** Successful creation of Event, Client, and Venue records, alongside dynamic many-to-many vendor assignments via the Event_Vendor__c junction object.
- **Programmatic Validation:** Real-time stress testing of the PreventDoubleBooking Apex Trigger and VenueStatusHelper class, confirming the system actively blocks scheduling conflicts and accurately toggles venue availability.
- **Workflow Automation:** Verification of the 3-day pre-event client reminder Flows and the multi-step Event Cancellation Approval Process, ensuring correct email routing and status updates.
- **Asynchronous Processing:** Validation of the nightly Batch Apex and Scheduled Jobs, confirming the automated transition of past events to a 'Completed' status without violating Salesforce governor limits.

- **Analytics Rendering:** Confirmation that the Event Planner Lightning App, Reports, and Operations Dashboards accurately consume and visualize real-time database metrics.

8.SYSTEM ADVANTAGES & DISADVANTAGES

Strategic Advantages

- **Centralized Operational Hub:** By unifying Events, Clients, Vendors, Venues, and Feedback into a single Salesforce relational model, the system eliminates data silos and spreadsheet dependency.
- **Maximized Efficiency via Automation:** Declarative Flows and scheduled background jobs automate routine administrative burdens, drastically reducing manual data entry and communication delays.
- **Logistical Conflict Prevention:** The deployment of custom Apex logic provides a programmatic safeguard against human error, actively protecting the business from the financial and reputational damage of double-booked venues.
- **Strict Operational Governance:** The structured Approval Process ensures that revenue-impacting decisions, such as event cancellations, require mandatory managerial oversight.
- **Zero-Trust Security Architecture:** A robust matrix of Profiles, Role Hierarchies, Permission Sets, and a 'Private' OWD guarantees that sensitive corporate and client data is strictly siloed based on the Principle of Least Privilege.
- **Real-Time Business Intelligence:** Executive Dashboards provide leadership with immediate visual insights into seasonal event volume, vendor utilization, and overall customer satisfaction.

System Limitations & Disadvantages

- **Development Environment Constraints:** The current architecture was engineered within a Salesforce Developer Edition org. While functionally complete, it is isolated from a live production environment.
- **Deployment Complexity:** Transitioning this framework to an enterprise production org requires specialized DevOps tooling, such as Salesforce DX, Change Sets, or third-party CI/CD pipelines (e.g., Copado, Gearset).

- **Technical Skill Barrier:** The reliance on programmatic solutions (Apex Classes, Triggers) and complex declarative automation (Flows) requires the organization to maintain access to certified Salesforce Administrators and Developers for ongoing troubleshooting.
- **Maintenance Overhead:** As business logic evolves, existing Validation Rules, Sharing Settings, and Flow criteria will require continuous monitoring and recalibration to prevent technical debt.

9.CONCLUSION

The EventForce Management System serves as a definitive blueprint for executing a digital transformation within the event planning sector. By transitioning from fragmented, manual workflows to a highly automated Salesforce CRM infrastructure, the project successfully establishes a single source of truth across the entire event lifecycle.

This implementation demonstrates the powerful synergy between Salesforce's declarative capabilities and custom programmatic logic. Validation Rules, Formula Fields, and Flows handle the immediate user interface and communication layers, while robust Apex Triggers and Batch classes operate in the background to enforce complex business rules, such as double-booking prevention and asynchronous status updates.

Furthermore, the project underscores the critical importance of a hermetically sealed security model. By combining a locked-down Organization-Wide Default baseline with lateral Sharing Rules, the system proves that cross-departmental collaboration can be achieved without sacrificing data privacy. While developed in a sandbox environment, the EventForce architecture directly reflects enterprise-grade implementation practices, providing a scalable, secure, and highly efficient foundation ready for real-world business operations.

10. FUTURE SCOPE

To transition the EventForce implementation from a foundational CRM to a comprehensive, enterprise-ready platform, the following strategic enhancements are recommended for future deployment phases:

- **Production Deployment & DevOps:** Execute a formal migration of the metadata and data model from the Developer Org to a live Production environment utilizing automated CI/CD deployment pipelines.
- **Salesforce Experience Cloud Integration:** Develop a secure, external-facing web portal that empowers Vendors to log in, view their assigned events, update their availability, and submit invoices directly into the CRM.
- **Third-Party API Integrations:** Integrate the Salesforce architecture with external business systems, such as financial gateways (e.g., Stripe/QuickBooks) for automated billing, and marketing platforms (e.g., Marketing Cloud) for broad client outreach.
- **CRM Analytics (Einstein):** Elevate the reporting capabilities by introducing AI-driven predictive analytics to forecast seasonal revenue trends, predict venue utilization bottlenecks, and perform sentiment analysis on client feedback.
- **Omnichannel Support & Mobile Optimization:** Refine the Lightning App interface specifically for the Salesforce Mobile Application, allowing on-site Event Coordinators to scan tickets, update statuses, and log vendor issues in real-time from the venue floor.
- **Advanced Vendor SLA Tracking:** Introduce Service Level Agreement (SLA) tracking and automated performance scoring mechanisms to dynamically rank vendors based on historical feedback and reliability.

11.APPENDIX

- **A. Relational Data Model:** Custom Objects (Event__c, Client__c, Vendor__c, Venue__c, Feedback__c) and Junction Architecture (Event_Vendor__c).
- **B. Declarative Automation:** Record-Triggered Flows (Client Reminders), Validation Rules (Data Integrity), and multi-step Approval Processes (Cancellations).
- **C. Programmatic Business Logic:** Apex Triggers (PreventDoubleBooking), Apex Helper Classes (VenueStatusHelper), and Asynchronous Operations (Batch/Scheduled Apex for Event Completion).
- **D. Security Architecture:** Custom Profiles, Role Hierarchy design, modular Permission Sets, 'Private' Organization-Wide Defaults (OWD), and Criteria-Based Sharing Rules.
- **E. Data Governance:** Pre-validation logic and relational data migration execution utilizing the Salesforce Data Import Wizard.

- **F. User Interface & Business Intelligence:** Custom Event Planner Lightning App configuration, aggregated Operational Reports, and real-time Executive Dashboards.
- **G. Quality Assurance:** Documentation detailing configuration auditing, role-based security testing, Apex test class coverage, and workflow validation.

THANK YOU